
FPGA AI SUITE

1.1 Inicio

El software *FPGA AI Suite* es un conjunto de herramientas desarrolladas por Intel para acelerar la implementación de modelos de inteligencia artificial y machine learning en FPGAs con la finalidad de optimizar y ejecutar redes neuronales.

Para usar el FPGA AI Suite se recomienda estar familiarizado con la distribución de Intel de las herramientas de OpenVINO.

1.2 Componentes del FPGA AI Suite

El FPGA AI Suite consiste de varios componentes para ayudar a habilitar modelos de inteligencia artificial en FPGAs de Altera. Consiste de los siguientes componentes:

- **IP**

El FPGA AI Suite IP es un bloque de propiedad intelectual precompilado con interfaces AXI que se pueden instanciar directamente en un diseño RTL en un sistema FPGA genérico.

- **Compiler (comando `dla_compiler`)**

El FPGA AI Suite compiler es una herramienta multipropósito que se puede usar para las siguientes tareas con el FPGA AI Suite :

- **Generación de arquitecturas**

Uso del compilador para generar una parametrización del IP que es optimizada para un modelo o grupo de modelos de machine learning (ML), tratando de ajustar el bloque IP del FPGA AI Suite dentro de un límite determinado de recursos.

- **Estimación del rendimiento del IP**

Se usa el compilador para obtener una estimación del rendimiento del bloque AI IP para una parametrización dada.

- **Estimación del consumo de recursos del FPGA IP**

Se usa el compilador para obtener una estimación de los recursos (ALMs, bloques M20k, y DSPs) del FPGA requeridos para una parametrización IP dada.

- **Creación de los gráficos compilados AOT (ahead-of-time)**

Se crea una versión compilada de un modelo ML que incluye las instrucciones necesarias para controlar el IP en el FPGA. El binario compilado incluye tanto las instrucciones para controlar el IP durante la inferencia y los pesos del modelo. Este modelo compilado es apropiado para el uso del diseño de ejemplo.

- **Herramienta de generación de IP**

La herramienta de generación del FPGA AI Suite personaliza el FPGA AI Suite IP basado en un archivo de arquitectura de entrada. El IP generado se coloca en una librería IP que se puede importar a un diseño FPGA diseñado con Platform Designer, usar directamente en un diseño RTL puro, o ambos.

El siguiente diagrama muestra la conexión entre los dos componentes principales del FPGA AI Suite , el compilador y el IP.

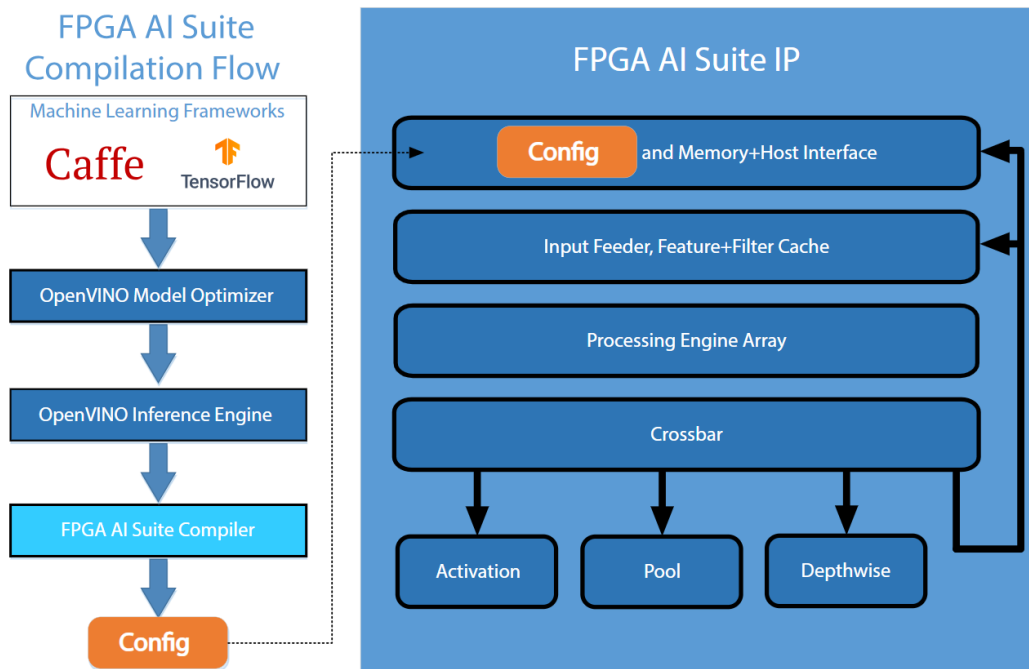


Figura 1.1: Conexión entre el FPGA AI Suite compiler y el FPGA AI Suite IP.

1.3 Tutorial de inicio rápido

En esta sección se describen los pasos para ejecutar aplicación de demostración y acelerar la inferencia usando el diseño de ejemplo.

- Se genera un flujo de datos y se carga una imagen preconfigurada en una tarjeta SD a la tarjeta de desarrollo.
- Ejecución de la herramienta [dla_benchmark](#) de la rutina de ejemplo en la tarjeta de desarrollo, la cual usa el modelo memory-to-memory (M2M).
- Ejecución de la aplicación de transmisión de imágenes que transmite los datos del procesador del HPS a la FPGA, de manera que imita cómo los datos son transmitidos desde cualquier otra fuente de ingreso (como Ethernet, HDMI o MIPI). Esta aplicación de transmisión de imágenes usa el modelo streaming-to-memory (S2M).

Para usar el software FPGA AI Suite se deben seguir los siguientes pasos:

1. Conectar el cable USB 2.0 tipo B de la tarjeta a la estación de trabajo.
2. Conectar la tarjeta hija HPS IO48 OOB-E al puerto **QSPI**.
3. Conectar el cable mini USB de la tarjeta hija a la estación de trabajo, y el cable Ethernet de la red local a la tarjeta hija.
4. Revisar que la configuración de la tarjeta sea correcta.
5. Revisa el estatus de los módulos externos: cable MXPM/QSFP/QSFPDD/FMC/DIMM
6. Encender la tarjeta.
7. En la estación de trabajo, en una nueva terminal ejecuta el comando `aisuit_init`.
8. Preparar el modelo de OpenVINO y compilar los gráficos que se van a cargar a la tarjeta.
9. Preparar el conjunto de imágenes.
10. Programar la FPGA y realizar las inferencias.

1.3.1 Configuración de inicio

Para iniciar con el ejemplo es necesario inicializar ciertas variables de entorno para OpenVino y FPGA AI Suite, las cuales por facilidad se agregaron al archivo de texto `~/Desktop/Variables_AI_Suite.txt`, el cual contiene las instrucciones para establecer las variables de entorno necesarias y las variables para apuntar al directorio de trabajo, que está ubicado en `~/coredla_work`.

```
# Configuración de variables de entorno para usar el FPGA AI Suite

# Variables de entorno de OpenVino
source ~/build-openvino-dev/openvino_env/bin/activate
unset python_version
source /opt/intel/openvino_2023/setupvars.sh

# Variables de entorno de FPGA AI Suite
source /opt/intel/fpga_ai_suite_2024.3/dla/setupvars.sh

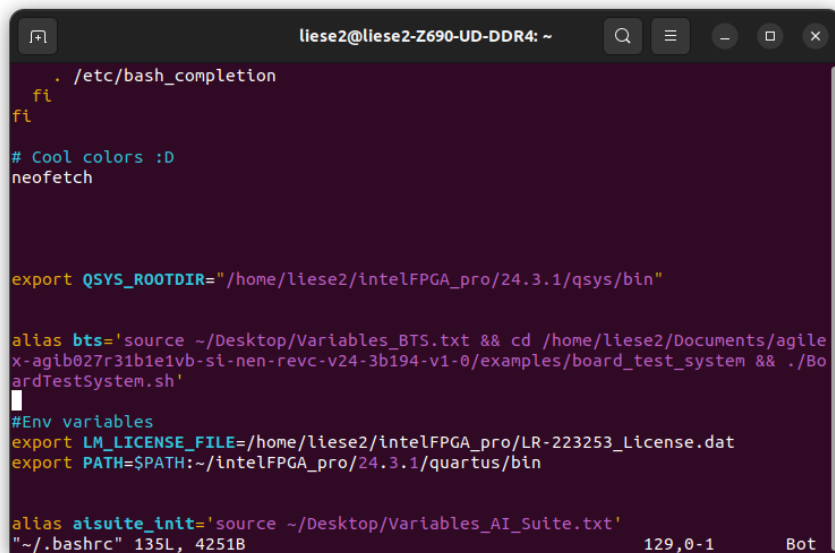
# Directorio de trabajo e inicialización de variables
cd ~/coredla_work
source ./coredla_work.sh
cd ~
```

Es importante saber que para trabajar con el FPGA AI Suite se tienen los directorios que se listan en la [tabla 1.1](#) y la función de cada uno de ellos.

Directorio	Descripción
<code>\$COREDLA_ROOT/</code>	Directorio base
<code>\$COREDLA_ROOT/bin</code>	Herramientas ejecutables (comando <code>dla_compiler</code>)
<code>\$COREDLA_ROOT/lib</code>	Librerías (compiler API, OpenVINO plugins, etc.)
<code>\$COREDLA_ROOT/inc</code>	Archivos API de encabezado públicos
<code>\$COREDLA_ROOT/util</code>	Módulos de entorno de ejecución adicionales
<code>\$COREDLA_ROOT/thirdparty</code>	Paquetes de terceros usados por el entorno de ejecución
<code>\$COREDLA_ROOT/area_model</code>	Archivos de datos
<code>\$COREDLA_ROOT/demo</code>	Imágenes de ejemplo y bitstreams
<code>\$COREDLA_ROOT/example_architectures</code>	Archivos de arquitectura, cada uno define una parametrización específica del IP
<code>\$COREDLA_ROOT/example_ip_cores</code>	Núcleos IP generados correspondientes a las arquitecturas de ejemplo
<code>\$COREDLA_ROOT/fpga</code>	Archivos de código fuente usados por el generador IP
<code>\$COREDLA_ROOT/licenses</code>	Información de la licencia para el FPGA AI Suite y las librerías incluidas
<code>\$COREDLA_ROOT/platform</code>	Directorio fuente BSP
<code>\$COREDLA_ROOT/runtime</code>	Directorio con demos de OpenVINO y entornos de ejecución de bajo nivel para la interfaz del FPGA
<code>\$COREDLA_ROOT/dla_plugin</code>	Directorio para la compilación heterogénea de OpenVINO e integración de inferencias con el FPGA

Tabla 1.1: Directorios en los que se encuentran los modelos compilados y los archivos de demostración.

Se asignó un alias para realizar un llamado desde terminal con el comando `aisuite_init`, que ejecuta las instrucciones anteriores y se define en el archivo `~/.bashrc`.



```

. /etc/bash_completion
fi
fi

# Cool colors :D
neofetch

export QSYS_ROOTDIR="/home/lieese2/intelFPGA_pro/24.3.1/qsys/bin"

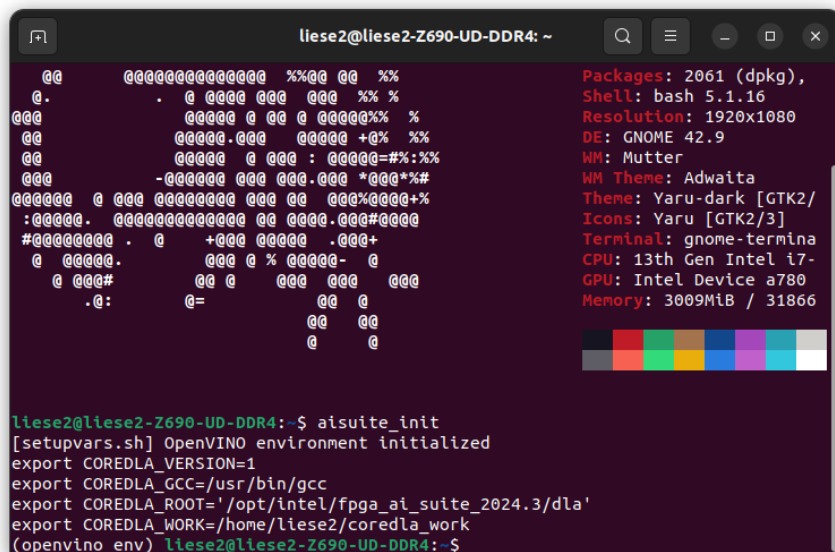
alias bts='source ~/Desktop/Variables_BTS.txt && cd /home/lieese2/Documents/agile
x-agib027r31b1e1vb-si-nen-revc-v24-3b194-v1-0/examples/board_test_system && ./Bo
ardTestSystem.sh'

#Env variables
export LM_LICENSE_FILE=/home/lieese2/intelFPGA_pro/LR-223253_License.dat
export PATH=$PATH:~/intelFPGA_pro/24.3.1/quartus/bin

alias aisuite_init='source ~/Desktop/Variables_AI_Suite.txt'
"~/.bashrc" 135L, 4251B 129,0-1 Bot

```

Figura 1.2: Alias y variables definidos en ~/.bashrc



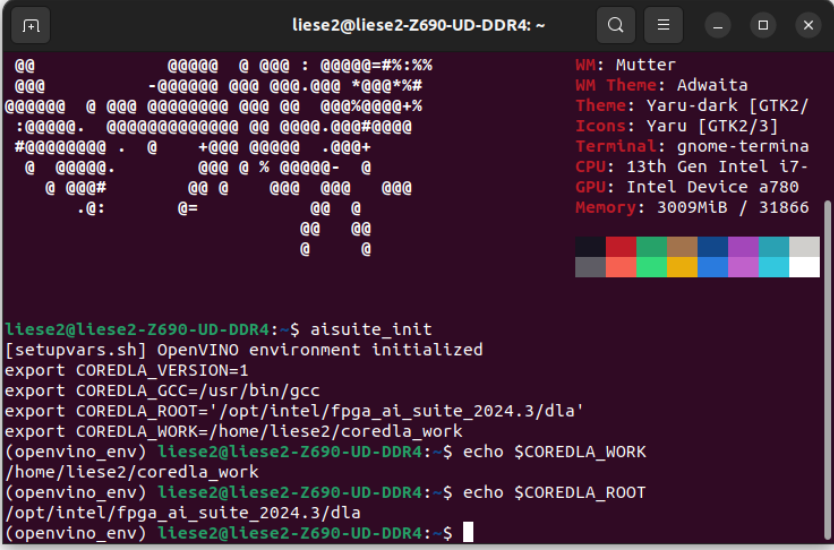
```

lieese2@lieese2-Z690-UD-DDR4: ~$ aisuite_init
[setupvars.sh] OpenVINO environment initialized
export COREDLA_VERSION=1
export COREDLA_ROOT=/usr/bin/gcc
export COREDLA_ROOT='/opt/intel/fpga_ai_suite_2024.3/dla'
export COREDLA_WORK=/home/lieese2/coredla_work
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$

```

Figura 1.3: Ejecución del comando `aisuite_init` para inicialización del entorno de trabajo y variables de entorno.

El script `coredla.work.sh` establece las variables de entorno `COREDLA_WORK` y `COREDLA_ROOT`, las cuales se usarán posteriormente y se puede verificar que se hayan inicializado correctamente desde terminal.



```

lieese2@lieese2-Z690-UD-DDR4: ~
@@          @@@@ @ @@@ : @@@@=#:%%
@@@         -@@@@@ @@@ @@@ *@@@*%#
@@@@@@ @ @@@ @@@@@@@@ @@@ @ @ @@@%@@@@+
:@@@@@. @@@@@@@@@@@@@ @ @@@@.@@@#@@@@
#@@@@@@@@ . @ +@@@ @@@@@ .@@@+
@ @@@@@. @@@ @ % @@@@@- @
@ @@@# @ @ @ @@@ @@@ @@@
.@: @= @ @ @ @
      @ @
      @ @
      @ @

WM: Mutter
WM Theme: Adwaita
Theme: Yaru-dark [GTK2/
Icons: Yaru [GTK2/3]
Terminal: gnome-terminal
CPU: 13th Gen Intel i7-
GPU: Intel Device a780
Memory: 3009MiB / 31866

lieese2@lieese2-Z690-UD-DDR4:~$ ailsuite_init
[setupvars.sh] OpenVINO environment initialized
export COREDLA_VERSION=1
export COREDLA_GCC=/usr/bin/gcc
export COREDLA_ROOT='/opt/intel/fpga_ai_suite_2024.3/dla'
export COREDLA_WORK=/home/lieese2/coredla_work
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ echo $COREDLA_WORK
/home/lieese2/coredla_work
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ echo $COREDLA_ROOT
/opt/intel/fpga_ai_suite_2024.3/dla
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$

```

Figura 1.4: Verificación para la inicialización de las variables del directorio de trabajo.

1.3.2 Cargar la imagen para tarjeta SD (.wic) a una tarjeta SD

Antes de ejecutar la demostración se debe crear una tarjeta SD booteable para la tarjeta de desarrollo. Se puede utilizar tanto la imagen precompilada u otra imagen diferente que se haya creado.

Nota: La imagen precompilada ya ha sido cargada a la tarjeta SD. Seguir a la sección [Preparando la tarjeta de desarrollo para el diseño de ejemplo de FPGA AI Suite](#). si no se va a cargar una nueva imagen a la tarjeta SD.

La imagen precompilada (.wic) se encuentra en el directorio `/opt/intel/fpga_ai_suite_2024.3/dla/demo/ed4/agx7_soc_s2m/sd-card/coredla-image-agilex7_dk_si_agi027fa.wic`

Para cargar la imagen a la tarjeta SD se deben seguir ciertos pasos:

1. Determinar cuál es el dispositivo asociado a la tarjeta SD en la estación de trabajo al ejecutar el siguiente comando antes y después de conectar la tarjeta SD a la estación de trabajo.

```
cat /proc/partitions
```

Las localizaciones típicas para la tarjeta SD son `/dev/sdb` o `/dev/sdc`. La siguiente instrucción usa `/dev/sdx` como localización de ejemplo de la tarjeta SD.

2. Usar el comando `dd` para cargar la imagen a la tarjeta SD de la siguiente manera:

```

wic_image=/opt/intel/fpga_ai_suite_2024.3/dla/demo/ed4/agx7_soc_s2m/sd-card/coredla-image-
agilex7_dk_si_agi027fa.wic
sudo umount /dev/sdx*
sudo dd if=$wic_image of=/dev/sdx bs=1M
sudo sync
sudo udisksctl power-off -b /dev/sdx

```

Después de que la imagen se haya cargado, inserte la tarjeta SD en la tarjeta IO48 00BE para conectarla a la tarjeta de desarrollo.

1.3.3 Preparando la tarjeta de desarrollo para el diseño de ejemplo de FPGA AI Suite

Confirmar que la tarjeta de desarrollo tenga la configuración y conexiones adecuadas

Se debe revisar la configuración como sigue:

1. Confirmar que la configuración de los DIP Switches de la tarjeta de desarrollo sea correcta. Para este ejemplo todos los DIP Switches deben tener la configuración por default, exceptuando a S9. La configuración se muestra a continuación.

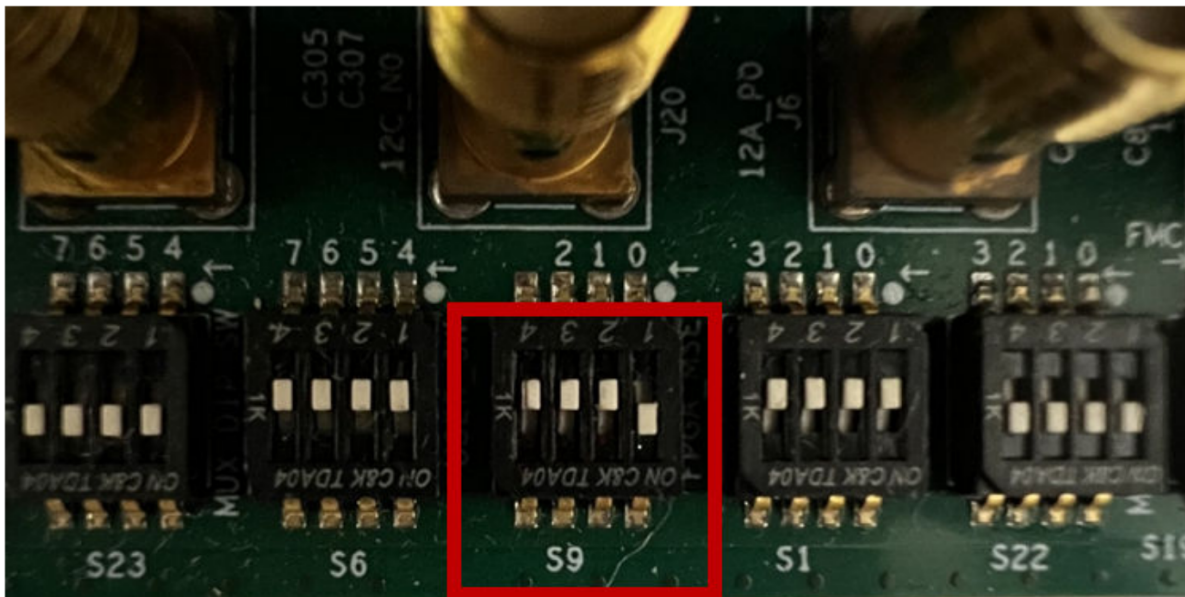


Figura 1.5: Configuración de los DIP Switches en la tarjeta de desarrollo.

2. La tarjeta HPS IO48 OOBEE debe estar instalada en el conector **QSPI J4** de la tarjeta de desarrollo y la tarjeta SD con la imagen Yocto programada debe estar instalada en la ranura correspondiente de la misma tarjeta hija.
3. Asegúrese que la memoria DDR4 x8 RDIMM está instalada en la ranura correspondiente.

Cuando la tarjeta de desarrollo esté conectada y configurada correctamente debería verse de la siguiente manera:

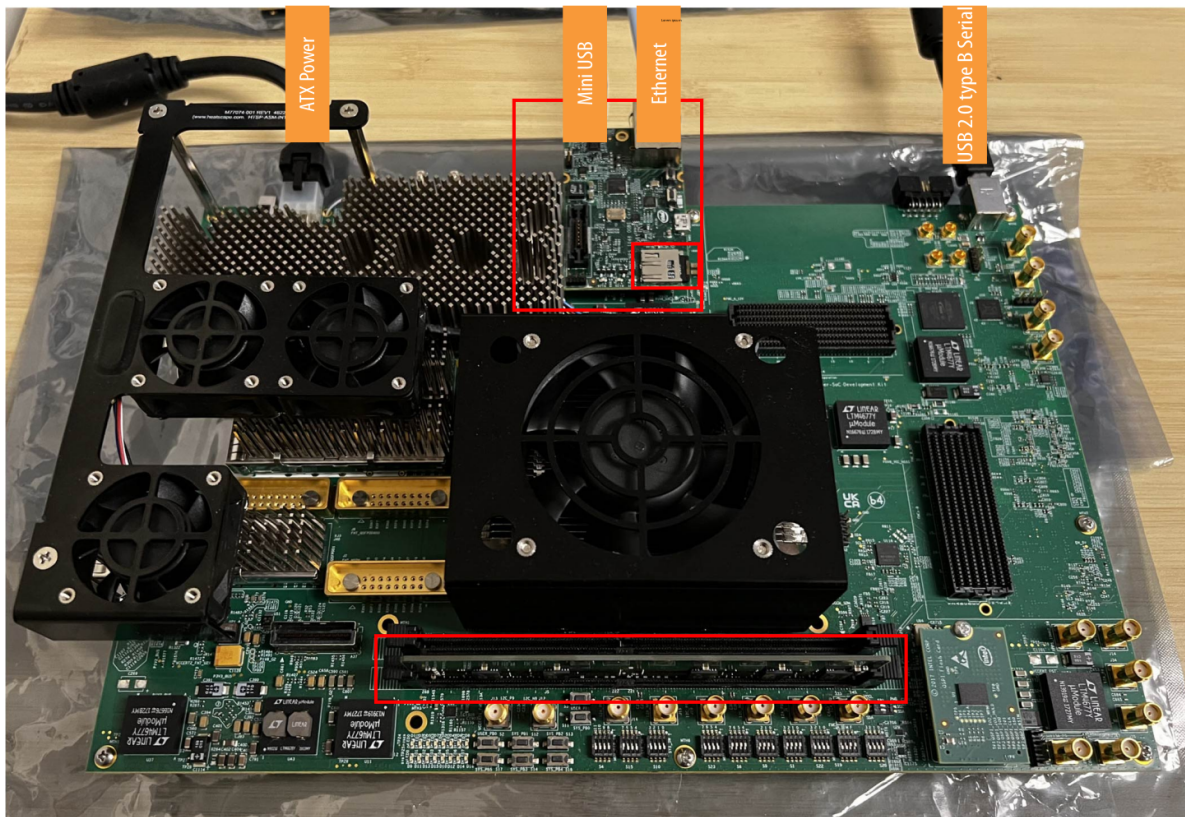


Figura 1.6: Conexión de la memoria DDR4 x8 RDIMM, tarjeta HPS IO48 OOB y tarjeta SD.

Las conexiones de la tarjeta de desarrollo sirven para lo siguiente:

- El conector USB 2.0 se usa para programar la FPGA.
- El conector Ethernet se usa para la transmisión rápida de datos al HPS¹.
- El conector micro USB se usa para monitorear la salida, y proveer las instrucciones de entrada al HPS durante la operación.

Programar la tarjeta de desarrollo con el archivo de configuración indirecta por JTAG (.jic)

Para configurar la tarjeta de desarrollo con el archivo de configuración indirecta por JTAG (.jic) se deben realizar los siguientes pasos:

1. Conecta la tarjeta de desarrollo a la estación de trabajo mediante el conector USB 2.0, como se muestra en la siguiente figura:

¹Para la conexión

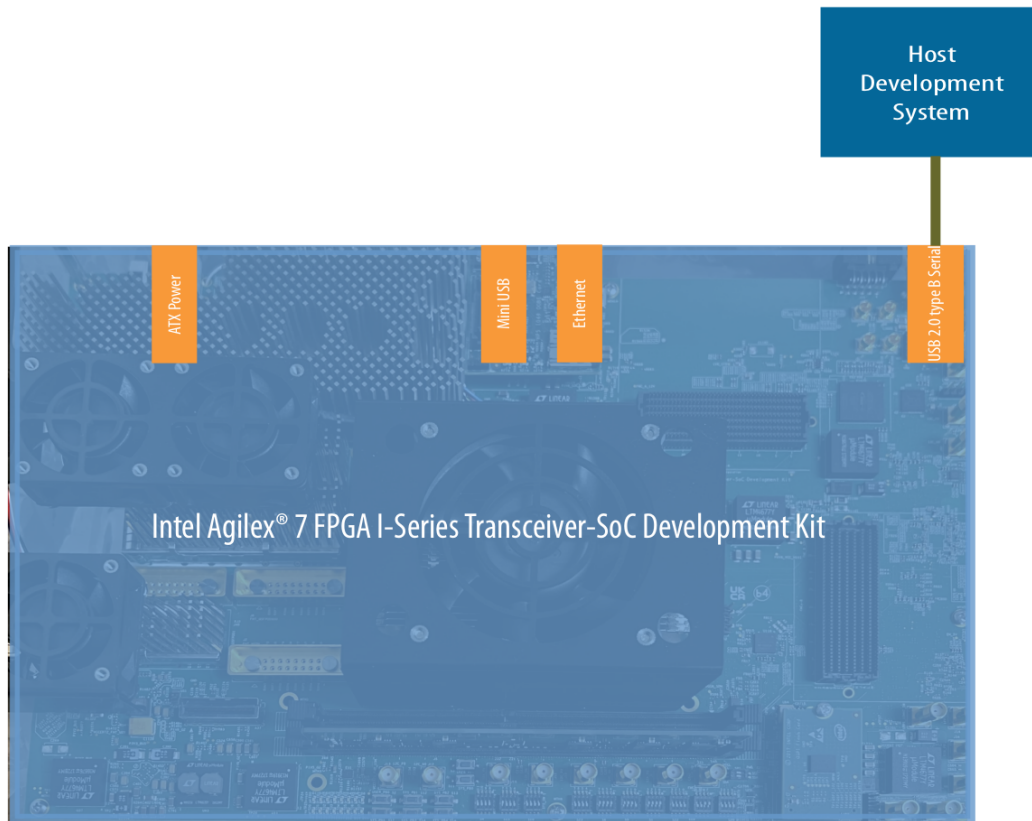


Figura 1.7: Conexión con el conector serial USB 2.0 tipo B a la computadora.

2. Programa el QSPI con el archivo `.jic` ejecutando los siguientes comandos en una terminal de la estación de trabajo.

```
cd $COREDLA_ROOT/demo/ed4/agx7_soc_s2m/sd-card/  
quartus_pgm -m jtag -o "pvi;u-boot-spl-dtb.hex.jic@<device_number>"
```

en donde `<device_number>` es 1 o 2, dependiendo si el HPS está encendido. Usa 1 si el HPS no está encendido, y 2 si ya lo está.

Tras unos cuantos minutos se mostrará la siguiente salida:

```

liese2@liese2-Z690-UD-DDR4: /opt/intel/fpga_ai_suite_2024.3/dla/demo/ed4/agx7_soc_s2m/sd-card
(openvino_env) liese2@liese2-Z690-UD-DDR4: /opt/intel/fpga_ai_suite_2024.3/dla/demo/ed4/agx7_soc_s2m/sd-card$ qua
rtus_pgm -m jtag -o pvi:u-boot-spl-dtb.hex.jic@2
Info: Running Quartus Prime Programmer
Info: Version 24.3.1 Build 102 01/14/2025 SC Pro Edition
Info: Copyright (C) 2025 Intel Corporation. All rights reserved.
Info: Your use of Intel Corporation's design tools, logic functions
Info: and other software and tools, and any partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Intel Program License
Info: Subscription Agreement, the Intel Quartus Prime License Agreement,
Info: the Intel FPGA IP License Agreement, or other applicable license
Info: agreement, including, without limitation, that your use is for
Info: the sole purpose of programming logic devices manufactured by
Info: Intel and sold by Intel or its authorized distributors. Please
Info: refer to the Intel FPGA Software License Subscription Agreements
Info: on the Quartus Prime software download page.
Info: Processing started: Fri May 2 16:59:24 2025
Info: System process ID: 10250
Info: Command: quartus_pgm -m jtag -o pvi:u-boot-spl-dtb.hex.jic@2
Info (213045): Using programming cable "Agilex I-Series SOC Dev Kit [1-6.3]"
Info (213011): Using programming file u-boot-spl-dtb.hex.jic with checksum 0x8B116B70 for device AG18027R31B02
Info (209060): Started Programmer operation at Fri May 2 16:59:29 2025
Info (18942): Configuring device index 2
Info (18943): Configuration succeeded at device index 1
Info (20103): Removed S10HPS from device index 1 after reconfiguration.
Info (19094): Erasing flash chip select 0 at device index 1
Info (19096): Programming flash chip select 0 at device index 1
Info (19097): Verifying flash chip select 0 at device index 1
Info (209011): Successfully performed operation(s)
Info (209061): Ended Programmer operation at Fri May 2 17:01:33 2025
Info: Quartus Prime Programmer was successful. 0 errors, 0 warnings
Info: Peak virtual memory: 2193 megabytes
Info: Processing ended: Fri May 2 17:01:33 2025
Info: Elapsed time: 00:02:09
(openvino_env) liese2@liese2-Z690-UD-DDR4: /opt/intel/fpga_ai_suite_2024.3/dla/de
(openvino_env) liese2@liese2-Z690-UD-DDR4: /opt/intel/fpga_ai_suite_2024.3/dla/demo/ed4/agx7_soc_s2m/sd-card$

```

Figura 1.8: Salida correspondiente después de ejecutar las líneas del código ??.

Conectar la tarjeta de desarrollo a la estación de trabajo

Conecta la tarjeta de desarrollo a la computadora mediante conexión Ethernet y USB UART como se muestra a continuación:

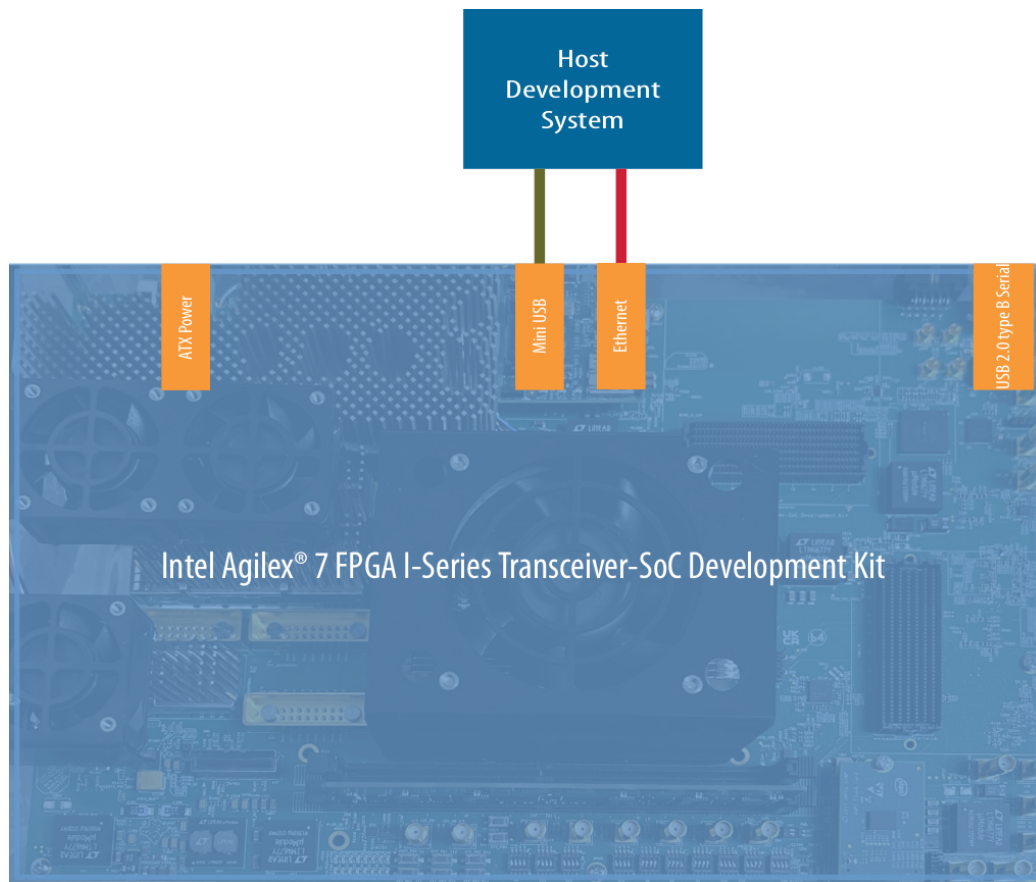


Figura 1.9: Conexión de Ethernet y mini USB de la tarjeta HPS IO48 OOB a la computadora.

1.3.4 Configuración de la conexión UART de la tarjeta de desarrollo

Una vez que nos aseguramos de que la tarjeta tiene todas las conexiones necesarias y la configuración correcta, procedemos a configurar la conexión UART de la tarjeta de desarrollo.

La FPGA Agilex 7 I-Series tiene un convertidor USB a serial en la tarjeta IO48 que le permite a la estación de trabajo detectarla como un puerto virtual serial.

Los parámetros de comunicación son los siguientes:

- Baud rate: 115200
- Parity: None
- Flow control: None
- Stop bits: 1

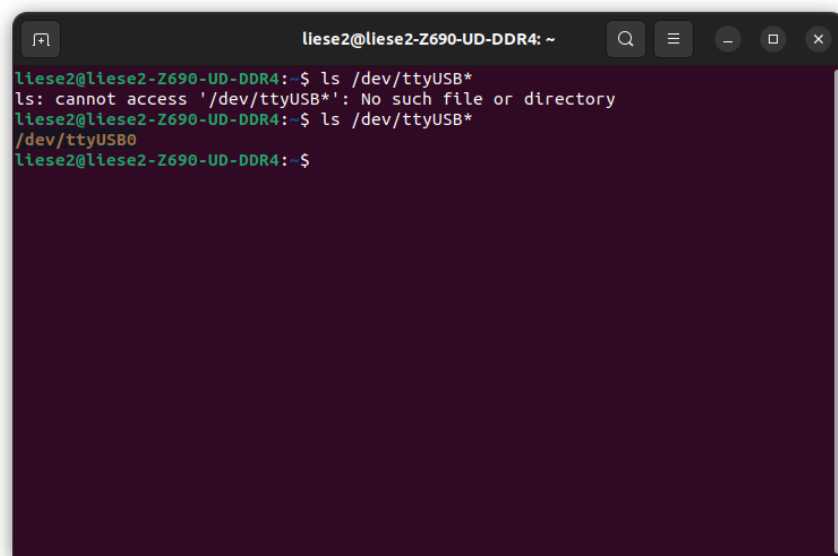
Estos parámetros se configuran en la herramienta *Minicom*, de la forma que sigue:

1. Determinar el nombre del dispositivo asociado con el puerto virtual serial en la estación de trabajo, que normalmente se llama `/dev/ttyUSB0`.

- a) Antes de conectar el cable mini USB a la tarjeta de desarrollo hay que determinar qué dispositivo USB serial está instalado con el siguiente comando:

```
ls /dev/ttyUSB*
```

- b) Conectar el cable mini USB de la tarjeta de desarrollo a la estación de trabajo.
- c) Confirmar nuevamente la nueva conexión del nuevo dispositivo con el comando `ls`.



```
lieese2@lieese2-Z690-UD-DDR4: ~  
lieese2@lieese2-Z690-UD-DDR4:~$ ls /dev/ttyUSB*  
ls: cannot access '/dev/ttyUSB*': No such file or directory  
lieese2@lieese2-Z690-UD-DDR4:~$ ls /dev/ttyUSB*  
/dev/ttyUSB0  
lieese2@lieese2-Z690-UD-DDR4:~$
```

Figura 1.10: Salida en terminal al ejecutar el comando anterior con el cable mini USB desconectado y luego conectado.

2. Si la aplicación Minicom no está instalada en la estación de trabajo hay que instalarla. En Ubuntu esto se realiza ejecutando el siguiente comando en una terminal:

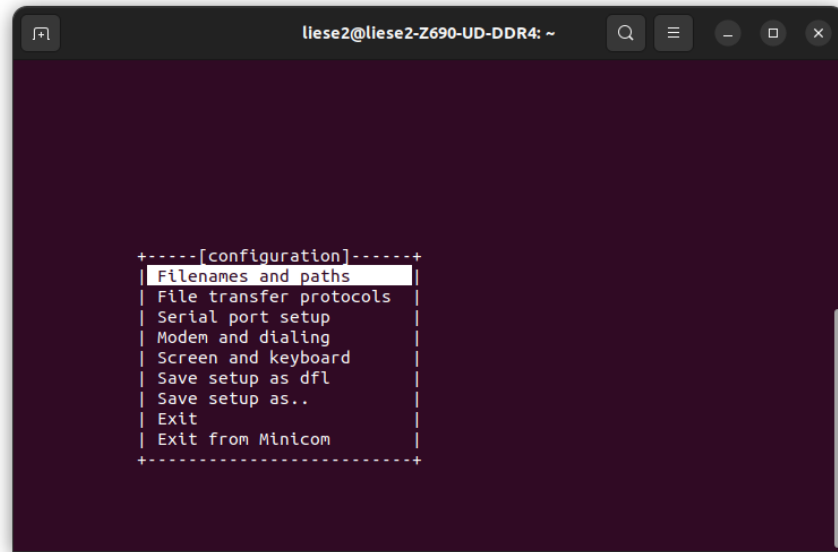
```
sudo apt-get install minicom
```

3. Configura Minicom de la siguiente manera:

- a) inicia Minicom con el siguiente comando:

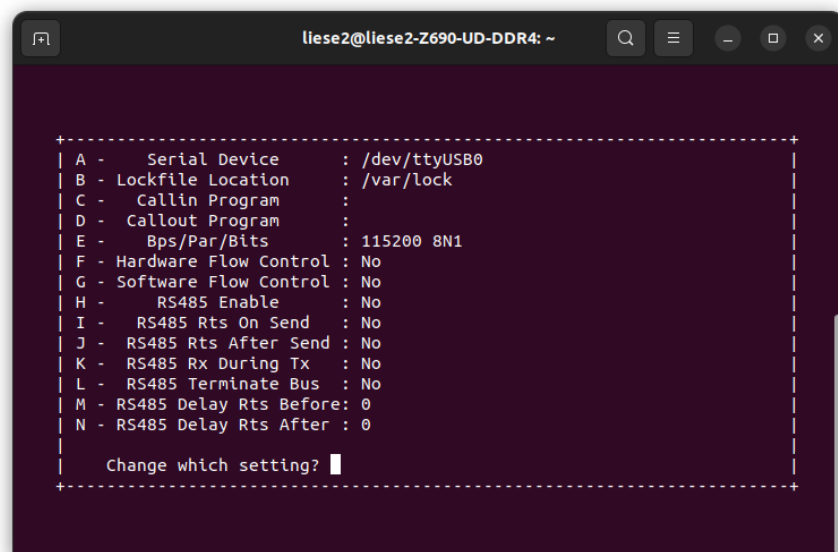
```
sudo minicom -s
```

Nota: el comando anterior es para modificar la configuración. Si no se necesita realizar ninguna configuración se puede ejecutar directamente sin el argumento `-s`.

Figura 1.11: Vista de las configuraciones de la herramienta *Minicom*.

b) En **Serial Port Setup** configurar de la siguiente forma:

- Serial Device: **/dev/ttyUSB0** (cambiar si el nombre es diferente del obtenido anteriormente).
- Bps/Par/Bits: **115200 8N1**
- Hardware Flow Control: **No**
- Software Flow Control: **No**

Figura 1.12: Menú de configuración de *Minicom* para el puerto serial.

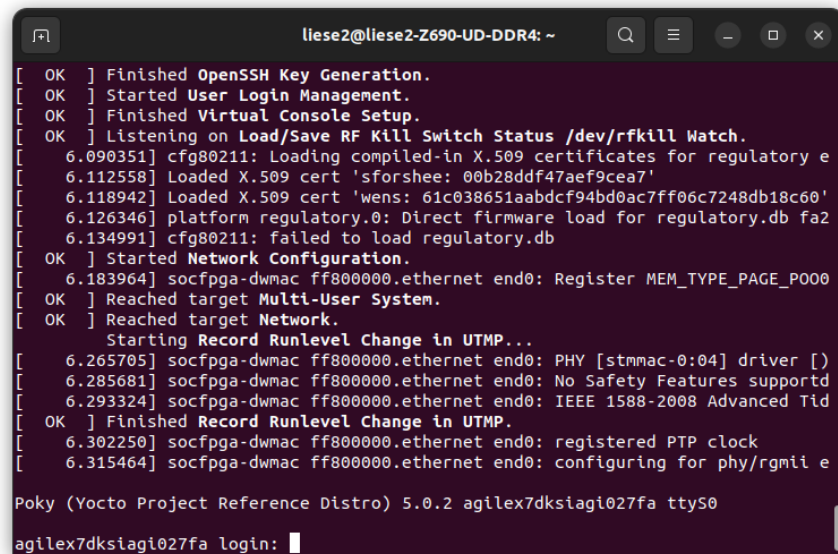
Presionar **Esc** para regresar a la configuración principal.

c) Selecciona **Save Setup as dfl** para guardar la configuración y salir seleccionando **Exit**.

1.3.5 Determinando la dirección IP de la tarjeta de desarrollo

Para determinar la dirección IP de la tarjeta de desarrollo:

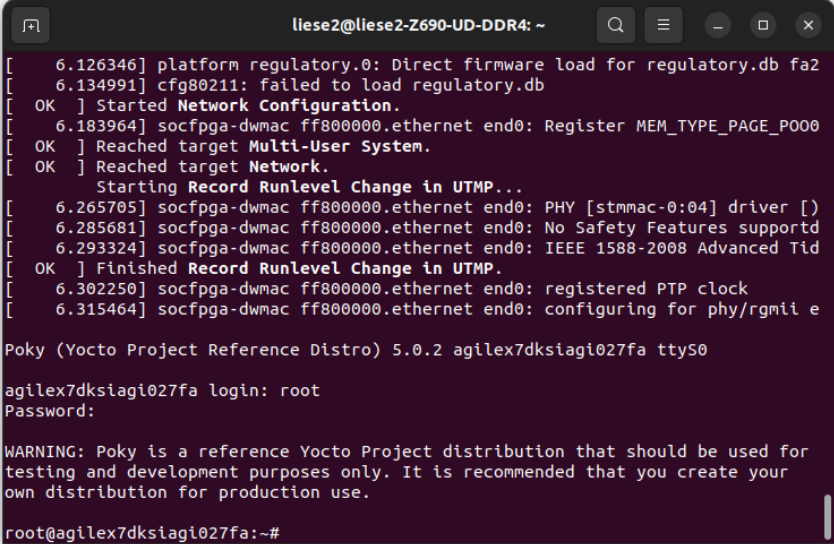
1. Después de configurar Minicom debemos encender la tarjeta de desarrollo, tras lo cual en terminal se configurará la conexión entre la estación de trabajo y la tarjeta de desarrollo. Si todo se realizó correctamente la terminal se detendrá en una ventana de login.



```
lieze2@lieze2-Z690-UD-DDR4: ~  
[ OK ] Finished OpenSSH Key Generation.  
[ OK ] Started User Login Management.  
[ OK ] Finished Virtual Console Setup.  
[ OK ] Listening on Load/Save RF Kill Switch Status /dev/rfkill Watch.  
[ 6.090351] cfg80211: Loading compiled-in X.509 certificates for regulatory e  
[ 6.112558] Loaded X.509 cert 'sforshee: 00b28ddf47aef9cea7'  
[ 6.118942] Loaded X.509 cert 'wens: 61c038651aabdcf94bd0ac7ff06c7248db18c60'  
[ 6.126346] platform regulatory.0: Direct firmware load for regulatory.db fa2  
[ 6.134991] cfg80211: failed to load regulatory.db  
[ OK ] Started Network Configuration.  
[ 6.183964] socfpga-dwmac ff800000.ethernet end0: Register MEM_TYPE_PAGE_P000  
[ OK ] Reached target Multi-User System.  
[ OK ] Reached target Network.  
Starting Record Runlevel Change in UTMP...  
[ 6.265705] socfpga-dwmac ff800000.ethernet end0: PHY [stmmac-0:04] driver []  
[ 6.285681] socfpga-dwmac ff800000.ethernet end0: No Safety Features supportd  
[ 6.293324] socfpga-dwmac ff800000.ethernet end0: IEEE 1588-2008 Advanced Tid  
[ OK ] Finished Record Runlevel Change in UTMP.  
[ 6.302250] socfpga-dwmac ff800000.ethernet end0: registered PTP clock  
[ 6.315464] socfpga-dwmac ff800000.ethernet end0: configuring for phy/rgmii e  
  
Poky (Yocto Project Reference Distro) 5.0.2 agilex7dksiagi027fa ttyS0  
agilex7dksiagi027fa login: █
```

Figura 1.13: Ventana de login para acceder al sistema de la tarjeta de desarrollo.

2. Acceder a la raíz de los archivos ingresando como usuario **root** y en contraseña ingresar **agilex7**.



```

liese2@liese2-Z690-UD-DDR4: ~
[ 6.126346] platform regulatory.0: Direct firmware load for regulatory.db fa2
[ 6.134991] cfg80211: failed to load regulatory.db
[ OK ] Started Network Configuration.
[ 6.183964] socfpga-dwmac ff800000.ethernet end0: Register MEM_TYPE_PAGE_POOL
[ OK ] Reached target Multi-User System.
[ OK ] Reached target Network.
Starting Record Runlevel Change in UTMP...
[ 6.265705] socfpga-dwmac ff800000.ethernet end0: PHY [stmmac-0:04] driver []
[ 6.285681] socfpga-dwmac ff800000.ethernet end0: No Safety Features supportd
[ 6.293324] socfpga-dwmac ff800000.ethernet end0: IEEE 1588-2008 Advanced Tid
[ OK ] Finished Record Runlevel Change in UTMP.
[ 6.302250] socfpga-dwmac ff800000.ethernet end0: registered PTP clock
[ 6.315464] socfpga-dwmac ff800000.ethernet end0: configuring for phy/rgmii e

Poky (Yocto Project Reference Distro) 5.0.2 agilex7dksiagi027fa ttyS0

agilex7dksiagi027fa login: root
Password:

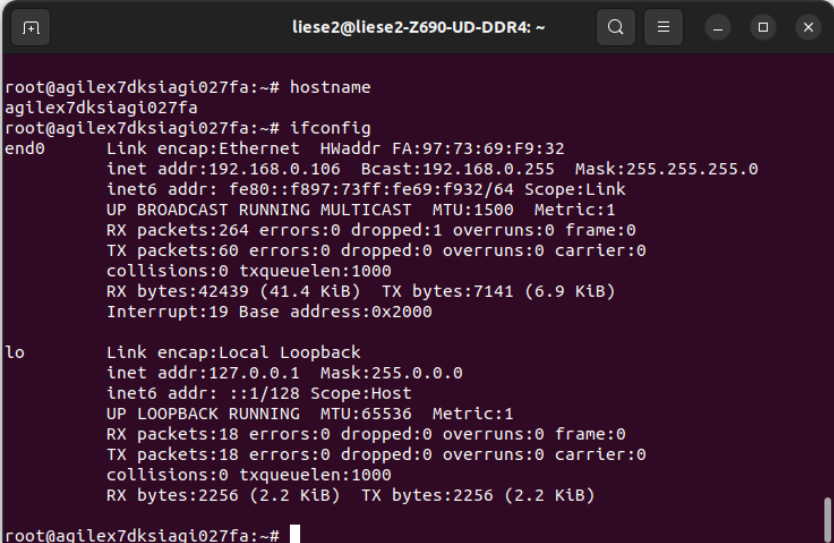
WARNING: Poky is a reference Yocto Project distribution that should be used for
testing and development purposes only. It is recommended that you create your
own distribution for production use.

root@agilex7dksiagi027fa:~#

```

Figura 1.14: Acceso a la raíz de archivos en Minicom.

- Dentro de `root` utilizamos el comando `hostname` para obtener el nombre de red de la tarjeta de desarrollo. También podemos obtener su dirección IP con el comando `ifconfig`.



```

root@agilex7dksiagi027fa:~# hostname
agilex7dksiagi027fa
root@agilex7dksiagi027fa:~# ifconfig
end0      Link encap:Ethernet  HWaddr FA:97:73:69:F9:32
          inet addr:192.168.0.106  Bcast:192.168.0.255  Mask:255.255.255.0
          inet6 addr: fe80::f897:73ff:fe69:f932/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:264 errors:0 dropped:1 overruns:0 frame:0
          TX packets:60 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:42439 (41.4 KiB)  TX bytes:7141 (6.9 KiB)
          Interrupt:19 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:18 errors:0 dropped:0 overruns:0 frame:0
          TX packets:18 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2256 (2.2 KiB)  TX bytes:2256 (2.2 KiB)

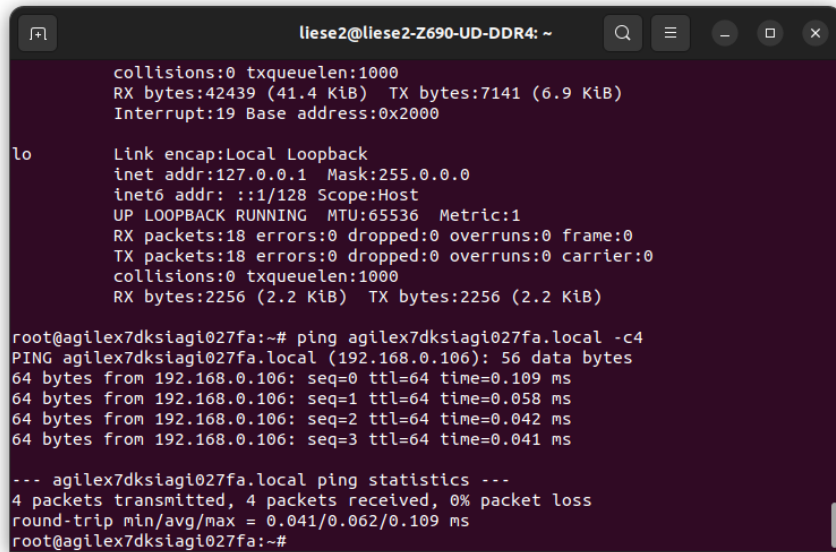
root@agilex7dksiagi027fa:~#

```

Figura 1.15: Nombre de red de la tarjeta y dirección IP.

Nota: Se va a necesitar este nombre de red (`agilex7dksiagi027fa`) más tarde para establecer una conexión SSH a la tarjeta de desarrollo.

- Comprobar que tenemos conexión de la tarjeta de desarrollo a la estación de trabajo mediante el comando `ping`. Cuando se use este comando debemos agregar `.local` al nombre de red de la tarjeta de desarrollo.



```

lliese2@lliese2-Z690-UD-DDR4: ~
collisions:0 txqueuelen:1000
RX bytes:42439 (41.4 KiB) TX bytes:7141 (6.9 KiB)
Interrupt:19 Base address:0x2000

lo    Link encap:Local Loopback
      inet addr:127.0.0.1 Mask:255.0.0.0
      inet6 addr: ::1/128 Scope:Host
      UP LOOPBACK RUNNING MTU:65536 Metric:1
      RX packets:18 errors:0 dropped:0 overruns:0 frame:0
      TX packets:18 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:2256 (2.2 KiB) TX bytes:2256 (2.2 KiB)

root@agilex7dkslgi027fa:~# ping agilex7dkslgi027fa.local -c4
PING agilex7dkslgi027fa.local (192.168.0.106): 56 data bytes
64 bytes from 192.168.0.106: seq=0 ttl=64 time=0.109 ms
64 bytes from 192.168.0.106: seq=1 ttl=64 time=0.058 ms
64 bytes from 192.168.0.106: seq=2 ttl=64 time=0.042 ms
64 bytes from 192.168.0.106: seq=3 ttl=64 time=0.041 ms

--- agilex7dkslgi027fa.local ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.041/0.062/0.109 ms
root@agilex7dkslgi027fa:~#

```

Figura 1.16: Comprobación de conexión entre la tarjeta de desarrollo y la estación de trabajo.

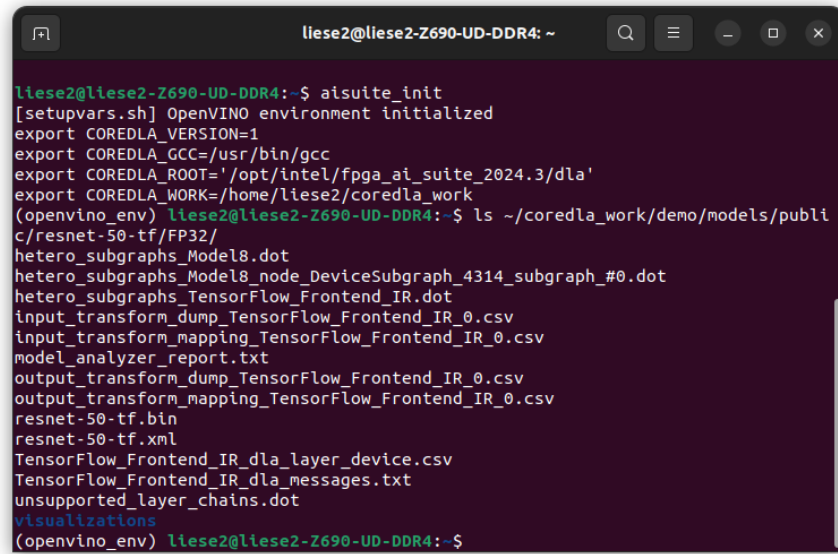
Se puede utilizar el nombre de red de la tarjeta cuando se necesita transferir archivos al sistema en ejecución agregando `.local` al nombre de red de la tarjeta.

1.3.6 Agregando los gráficos compilados (archivos AOT) a la tarjeta SD

Ya establecida la conexión debemos agregar los gráficos compilados (archivos AOT) a la tarjeta SD, estos archivos contienen instrucciones para que la IP de FPGA AI Suite se ejecute con el objetivo de realizar la inferencia. El modelo M2M y S2M requieren diferentes archivos AOT.

Para agregar los gráficos compilados a la tarjeta SD de la tarjeta de desarrollo realiza los siguientes pasos.

1. Crear el directorio `$COREDLA_WORK` si no se ha creado anteriormente.
2. Preparar el optimizador de OPenVINO Model Zoo y el optimizador de modelo.
3. Preparar el modelo.
4. Confirmar que el directorio `$COREDLA_WORK/demo/models/public/resnet-50-tf/FP32/` existe. Si no existe este directorio confirma que los tres primeros pasos se realizaron correctamente.



```
lieese2@lieese2-Z690-UD-DDR4: ~  
lieese2@lieese2-Z690-UD-DDR4:~$ aisuite_init  
[setupvars.sh] OpenVINO environment initialized  
export COREDLA_VERSION=1  
export COREDLA_GCC=/usr/bin/gcc  
export COREDLA_ROOT='/opt/intel/fpga_ai_suite_2024.3/dla'  
export COREDLA_WORK=/home/lieese2/coredla_work  
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ ls ~/coredla_work/demo/models/public/resnet-50-tf/FP32/  
hetero_subgraphs_Model8.dot  
hetero_subgraphs_Model8_node_DeviceSubgraph_4314_subgraph_#0.dot  
hetero_subgraphs_TensorFlow_Frontend_IR.dot  
input_transform_dump_TensorFlow_Frontend_IR_0.csv  
input_transform_mapping_TensorFlow_Frontend_IR_0.csv  
model_analyzer_report.txt  
output_transform_dump_TensorFlow_Frontend_IR_0.csv  
output_transform_mapping_TensorFlow_Frontend_IR_0.csv  
resnet-50-tf.bin  
resnet-50-tf.xml  
TensorFlow_Frontend_IR_dla_layer_device.csv  
TensorFlow_Frontend_IR_dla_messages.txt  
unsupported_layer_chains.dot  
visualizations  
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$
```

Figura 1.17: Elementos que contiene el directorio.

5. Compilar los gráficos.
6. Copiar los gráficos compilados a la tarjeta SD.

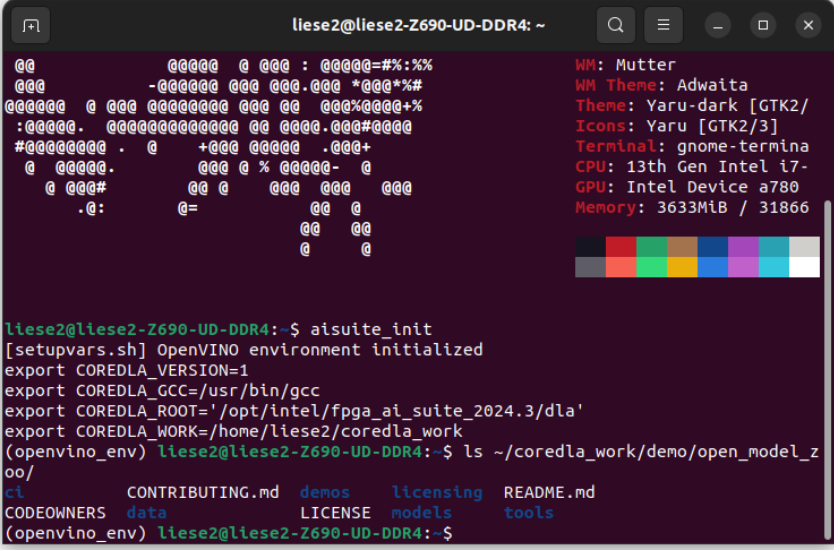
Preparando OpenVINO Model Zoo

Antes de ejecutar cualquier aplicación de demostración los modelos entrenados deben ser convertidos al formato de representación intermedia de OpenVINO (IR), `.xml/.bin` con el optimizador de modelos de OpenVINO.

Estas instrucciones asumen que existe una copia de OpenVINO Model Zoo 2022.3.1 en el directorio `$COREDLA_WORK/demo/open_model_zoo/`.

Para descargar una copia de Model Zoo, se ejecutan los siguientes comandos:

```
cd $COREDLA_WORK/demo  
git clone https://github.com/openvinotoolkit/open_model_zoo.git  
cd open_model_zoo  
git checkout 2022.3.1
```



```

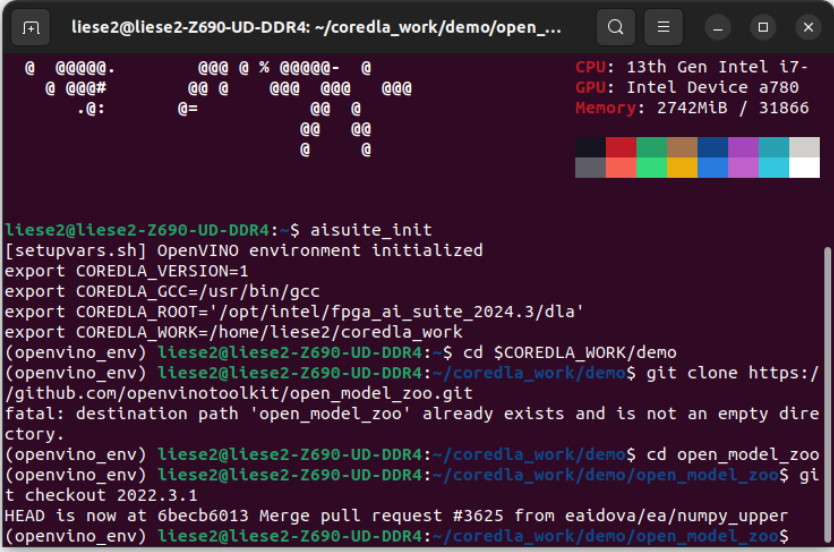
lieese2@lieese2-Z690-UD-DDR4: ~
@@          @@@@ @ @@@ : @@@@=#:%%
@@@         -@@@@@ @@@ @@@ *@@@*%#
@@@@@@ @ @@@ @@@@@@@@ @@@ @ @ @@@%@@@@+
:@@@@@. @@@@@@@@@@@@@@ @ @ @@@@.@@@@@@@@
#@@@@@@@@ . @ +@@@ @@@@@ .@@@+
@ @@@@@. @@@ @ % @@@@@- @
@ @@@# @ @ @ @@@ @@@ @@@
.@: @= @ @ @
      @ @
      @ @
      @

WM: Mutter
WM Theme: Adwaita
Theme: Yaru-dark [GTK2/
Icons: Yaru [GTK2/3]
Terminal: gnome-terminal
CPU: 13th Gen Intel i7-
GPU: Intel Device a780
Memory: 3633MiB / 31866

lieese2@lieese2-Z690-UD-DDR4:~$ aisuite_init
[setupvars.sh] OpenVINO environment initialized
export COREDLA_VERSION=1
export COREDLA_GCC=/usr/bin/gcc
export COREDLA_ROOT='/opt/intel/fpga_ai_suite_2024.3/dla'
export COREDLA_WORK=/home/lieese2/coredla_work
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ ls ~/coredla_work/demo/open_model_zoo/
cl      CONTRIBUTING.md  demos    licensing  README.md
CODEOWNERS  data      LICENSE  models    tools
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$

```

Figura 1.18: Elementos que contiene el directorio del modelo.



```

lieese2@lieese2-Z690-UD-DDR4: ~/coredla_work/demo/open...
@ @@@@@. @@@ @ % @@@@@- @
@ @@@# @ @ @ @@@ @@@ @@@
.@: @= @ @ @
      @ @
      @ @
      @

CPU: 13th Gen Intel i7-
GPU: Intel Device a780
Memory: 2742MiB / 31866

lieese2@lieese2-Z690-UD-DDR4:~$ aisuite_init
[setupvars.sh] OpenVINO environment initialized
export COREDLA_VERSION=1
export COREDLA_GCC=/usr/bin/gcc
export COREDLA_ROOT='/opt/intel/fpga_ai_suite_2024.3/dla'
export COREDLA_WORK=/home/lieese2/coredla_work
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ cd $COREDLA_WORK/demo
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo$ git clone https://github.com/openvinotoolkit/open_model_zoo.git
fatal: destination path 'open_model_zoo' already exists and is not an empty directory.
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo$ cd open_model_zoo
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo/open_model_zoo$ git checkout 2022.3.1
HEAD is now at 6becb6013 Merge pull request #3625 from eaidova/ea/numpy_upper
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo/open_model_zoo$

```

Figura 1.19: Descarga de una copia de OpenVINO Model Zoo.

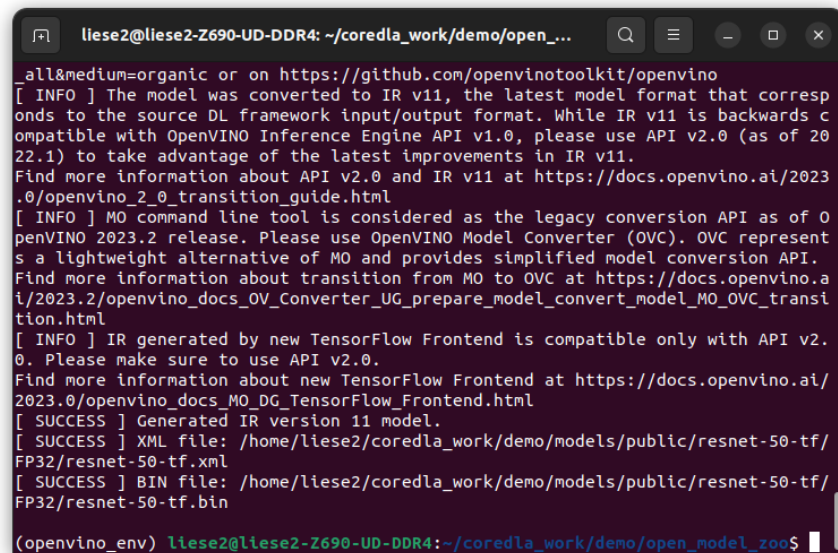
Preparando el modelo

El modelo debe ser convertido de un framework (como TensorFlow, Caffe, Pytorch) a archivos `.bin` y `.xml` antes de que el compilador de la FPGA AI Suite (`dla_compiler`) pueda incorporar el modelo.

Se deben seguir los siguientes comandos para descargar el modelo de la ResNet de TensorFlow y ejecutar el

optimizador del modelo.

```
source ~/build-openvino-dev/openvino_env/bin/activate
omz_downloader --name resnet-50-tf \
  --output_dir $COREDLA_WORK/demo/models/
omz_converter --name resnet-50-tf \
  --download_dir $COREDLA_WORK/demo/models/ \
  --output_dir $COREDLA_WORK/demo/models/
```



```
lieese2@lieese2-Z690-UD-DDR4: ~/coredla_work/demo/open...
_all&medium=organic or on https://github.com/openvinotoolkit/openvino
[ INFO ] The model was converted to IR v11, the latest model format that corresp
onds to the source DL framework input/output format. While IR v11 is backwards c
ompatible with OpenVINO Inference Engine API v1.0, please use API v2.0 (as of 20
22.1) to take advantage of the latest improvements in IR v11.
Find more information about API v2.0 and IR v11 at https://docs.openvino.ai/2023
.0/openvino_2_0_transition_guide.html
[ INFO ] MO command line tool is considered as the legacy conversion API as of O
penVINO 2023.2 release. Please use OpenVINO Model Converter (OVC). OVC represent
s a lightweight alternative of MO and provides simplified model conversion API.
Find more information about transition from MO to OVC at https://docs.openvino.a
i/2023.2/openvino_docs_OV_Converter_UG_prepare_model_convert_model_MO_OVC_transi
tion.html
[ INFO ] IR generated by new TensorFlow Frontend is compatible only with API v2.
0. Please make sure to use API v2.0.
Find more information about new TensorFlow Frontend at https://docs.openvino.ai/
2023.0/openvino_docs_MO_DG_TensorFlow_Frontend.html
[ SUCCESS ] Generated IR version 11 model.
[ SUCCESS ] XML file: /home/lieese2/coredla_work/demo/models/public/resnet-50-tf/
FP32/resnet-50-tf.xml
[ SUCCESS ] BIN file: /home/lieese2/coredla_work/demo/models/public/resnet-50-tf/
FP32/resnet-50-tf.bin
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo/open_model_zoo$
```

Figura 1.20: Salida en terminal al descargar el modelo entrenado.

El comando `omz_downloader` descarga el modelo entrenado de la carpeta `$COREDLA_WORK/demo/models`. Mientras que el comando `omz_converter` ejecuta el optimizador de modelo que convierte el modelo entrenado a una representación intermedia de archivos `.bin` y `.xml` en el directorio `$COREDLA_WORK/demo/models/public/resnet-50-tf/FP32/`.

El directorio `$COREDLA_WORK/demo/open_model_zoo/models/public/resnet-50-tf` contiene dos archivos prácticos que no aparecen en el directorio `$COREDLA_ROOT/demo/models`. Dichos archivos son:

- El archivo `README.md` el cual describe información a fondo sobre el modelo.
- El archivo `model.yml` el cual muestra la información detallada de la línea de comandos dada al optimizador de modelo (`mo.py`) cuando este convierte el modelo a archivos `.bin` y `.xml`.

Compilando los gráficos

La red descrita en los archivos `.xml` y `.bin` (creados con el optimizador de modelos) es compilada para un archivo de arquitectura específico del FPGA AI Suite usando el FPGA AI Suite Compiler. Este compilador compila la red y

se exporta a un archivo `.bin` con el formato requerido por el Motor de Inferencia de OpenVINO. Este archivo `.bin` contiene los parámetros de la red compilada para todos los dispositivos que lo requieran (FPGA, CPU, o ambos) junto con los pesos y sesgos.

El FPGA AI Suite Compiler también puede compilar los gráficos y dar mediciones de área estimada o rendimiento para un archivo de arquitectura dado o generar un archivo de arquitectura optimizado.

La imagen precompilada de la tarjeta SD (`.wic`) que viene incluida con la tarjeta de desarrollo usa el archivo de configuración de arquitectura IP `AGX7_Performance.arch`. Este archivo sirve para crear los archivos AOT para ambas variantes como se muestra a continuación.

Para crear el archivo AOT para la variante M2M (que usa la utilidad `dla_benchmark`) de la siguiente manera.

```
cd $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32
dla_compiler \
--march $COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
--network-file ./resnet-50-tf.xml \
--foutput-format=open_vino_hetero \
--o $COREDLA_WORK/demo/RN50_Performance_b1.bin \
--batch-size=1 \
--fanalyze-performance
```

Este comando especifica los siguientes archivos de entrada:

- El archivo `.arch` especifica la configuración de arquitectura del IP (incluyendo parámetros que afectarán cómo se comportará y qué tan grande será).
- El archivo `.xml` (junto con el `.bin`) describen a la red.
- El archivo `.bin` almacena ciertos parámetros junto con los pesos del modelo.

El compilador del FPGA AI Suite usa estos archivos para analizar el rendimiento estimado.

El archivo de salida `RN50_Performance_b1.bin` es la red compilada y contiene las instrucciones y pesos necesarios para controlar el FPGA AI Suite IP en el dispositivo FPGA. Este archivo de salida es específico al archivo `.arch` y el archivo `.xml`.

El gráfico de compilador muestra qué capas se ejecutan en el FPGA y cuáles se ejecutan en el CPU. Usando en paquete Graphviz, el archivo con formato dot se puede convertir en un archivo SVG llamado `ResNet-50.svg` que muestra cada capa. Se puede convertir el archivo dot en un archivo SVG usando el siguiente comando:

```
dot -Tsvg hetero_subgraphs_resnet-50-tf.dot -o ResNet-50.svg
```

```

liese2@liese2-Z690-UD-DDR4: ~/coredla_work/demo/models/public/resnet-50-tf/FP32
DDR CONFIG BUFFER SIZE = 0.0664062 MB
DDR FILTER BUFFER SIZE = 49.4229 MB
DDR INTERMEDIATE BUFFER SIZE = 3.0625 MB
NOTE: THIS ESTIMATE ASSUMES 1x I/O BUFFER. THE COREDLA RUNTIME DEFAULTS TO 5
TOTAL DDR TRANSFERS REQUIRED = 70.8179 MB
DDR FILTER READS REQUIRED = 52.6885 MB
DDR FEATURE READS REQUIRED = 11.1705 MB
DDR FEATURE WRITES REQUIRED = 6.89258 MB
NUMBER OF DDR FEATURE READS = 13
MINIMUM AVERAGE DDR BANDWIDTH REQUIRED = 10160.8 MB/s
-----
Performance Estimator Throughput Breakdown
Arch: kvec64xcvec32_i5x1_fp13agx_sb32768_xbark32_actk32_poolk4
Number of DLA instances = 1
Number of DDR Banks per DLA instance = 1
CoreDLA Target Fmax = 500 MHz
PE Target Fmax = 500 MHz
Batch Size = 1
PE-only Conv Throughput No DDR = 191 fps
PE-only Conv Throughput = 181 fps
Overall Throughput Inf PE Buf Depth (zero MPBW) = 180 fps
Overall Throughput Zero PE Buf Depth (zero MPBW) = 179 fps
Overall Throughput Inf PE Buf Depth = 174 fps
Overall Throughput Zero PE Buf Depth = 173 fps
-----
FINAL THROUGHPUT = 143.477 fps
FINAL THROUGHPUT PER FMAX (CoreDLA) = 0.286955 fps/MHz
FINAL THROUGHPUT PER FMAX (PE) = 0.286955 fps/MHz

```

Figura 1.21: Salida al crear el archivo AOT para la variante M2M.

Para crear el archivo AOT para la variante S2M (que usa la aplicación de transmisión de inferencia) se deben ejecutar los siguientes comandos.

```

cd $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32
dla_compiler \
--march $COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
--network-file ./resnet-50-tf.xml \
--foutput-format=open_vino_hetero \
--o $COREDLA_WORK/demo/RN50_Performance_no_folding.bin \
--batch-size=1 \
--fanalyze-performance \
--ffolding-option=0

```

```

liese2@liese2-Z690-UD-DDR4: ~/coredla_work/demo/models/public/resnet-50-tf/FP32
DDR FILTER BUFFER SIZE = 49.5518 MB
DDR INTERMEDIATE BUFFER SIZE = 3.82812 MB
NOTE: THIS ESTIMATE ASSUMES 1x I/O BUFFER. THE COREDLA RUNTIME DEFAULTS TO 5
TOTAL DDR TRANSFERS REQUIRED = 75.8902 MB
DDR FILTER READS REQUIRED = 53.3931 MB
DDR FEATURE READS REQUIRED = 14.7686 MB
DDR FEATURE WRITES REQUIRED = 7.6582 MB
NUMBER OF DDR FEATURE READS = 18
MINIMUM AVERAGE DDR BANDWIDTH REQUIRED = 9773.24 MB/s
-----
Performance Estimator Throughput Breakdown
Arch: kvec64xcvec32_15x1_fp13agx_sb32768_xbark32_actk32_poolk4
Number of DLA Instances = 1
Number of DDR Banks per DLA instance = 1
CoreDLA Target Fmax = 500 MHz
PE Target Fmax = 500 MHz
Batch Size = 1
PE-only Conv Throughput No DDR = 164 fps
PE-only Conv Throughput = 156 fps
Overall Throughput Inf PE Buf Depth (zero MPBW) = 156 fps
Overall Throughput Zero PE Buf Depth (zero MPBW) = 155 fps
Overall Throughput Inf PE Buf Depth = 150 fps
Overall Throughput Zero PE Buf Depth = 150 fps
-----
FINAL THROUGHPUT = 128.781 fps
FINAL THROUGHPUT PER FMAX (CoreDLA) = 0.257563 fps/MHz
FINAL THROUGHPUT PER FMAX (PE) = 0.257563 fps/MHz
(openvino_env) liese2@liese2-Z690-UD-DDR4: ~/coredla_work/demo/models/public/resnet-50-tf/FP32$

```

Figura 1.22: Salida al crear el archivo AOT para la variante S2M.

Después de ejecutar cualquiera de estas secuencias encontraremos los modelos compilados y los archivos de demostración en los siguientes directorios.

Modelos Compilados	<code>\$COREDLA_WORK/demo/RN50_Performance_b1.bin</code> <code>\$COREDLA_WORK/demo/RN50_Performance_no_folding.bin</code>
Imágenes de muestra	<code>\$COREDLA_WORK/demo/sample_images/</code>
Archivo de arquitectura	<code>\$COREDLA_ROOT/example_architectures/AGX7_Performance.arch</code>

Tabla 1.2: Directorios en los que se encuentran los modelos compilados y los archivos de demostración.

```

lieese2@lieese2-Z690-UD-DDR4: ~/coredla_work/demo/sample_images
Performance Estimator Throughput Breakdown
Arch: kvec64xcvec32_i5x1_fp13agx_sb32768_xbark32_actk32_poolk4
Number of DLA instances           = 1
Number of DDR Banks per DLA instance = 1
CoreDLA Target Fmax              = 500 MHz
PE Target Fmax                   = 500 MHz
Batch Size                       = 1
PE-only Conv Throughput No DDR   = 164 fps
PE-only Conv Throughput          = 156 fps
Overall Throughput Inf PE Buf Depth (zero MPBW) = 156 fps
Overall Throughput Zero PE Buf Depth (zero MPBW) = 155 fps
Overall Throughput Inf PE Buf Depth = 150 fps
Overall Throughput Zero PE Buf Depth = 150 fps
-----
FINAL THROUGHPUT = 128.781 fps
FINAL THROUGHPUT PER FMAX (CoreDLA) = 0.257563 fps/MHz
FINAL THROUGHPUT PER FMAX (PE) = 0.257563 fps/MHz
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo/models/public/resnet-50-tf/FP32$ cd ~
/coredla_work/demo/
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo$ ls
bitstreams  models          RN50_Performance_b1.bin  sample_images
ed4         open_model_zoo RN50_Performance_no_folding.bin sample_videos
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo$ cd sample_images/
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo/sample_images$ ls
224x224x32bpp      val_00000000.bmp  val_00000003.bmp  val_00000006.bmp  val_00000009.bmp
ground_truth.txt   val_00000001.bmp  val_00000004.bmp  val_00000007.bmp
TF_ground_truth.txt val_00000002.bmp  val_00000005.bmp  val_00000008.bmp
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~/coredla_work/demo/sample_images$

```

Figura 1.23: Directorios que contienen los modelos compilados y los archivos de demostración.

Preparando un set de imágenes

Esta sección describe cómo preparar un set de imágenes para clasificación de gráficos que requiere entradas de 224x224 y que ha sido entrenada con las clasificaciones de ImageNet. El FPGA AI Suite incluye ejemplos de imágenes de validación en el directorio `$COREDLA_ROOT/demo/sample_images/`.

Los contenidos son los siguientes:

<code>\$COREDLA_WORK/demo/sample_images/</code>	Directorio de imágenes de ejemplo. Se usa una convención de nombrado de archivos para dar un orden a las imágenes.
<code>\$COREDLA_WORK/demo/sample_images/ground_truth.txt</code>	Archivo ground truth, contiene las clases correctas.
<code>\$COREDLA_WORK/demo/sample_images/TF_ground_truth.txt</code>	Archivo ground truth apropiado para los gráficos del framework TensorFlow.

Tabla 1.3: Contenidos del directorio de ejemplo.

Los gráficos de clasificación de imágenes entrenados en el framework TensorFlow necesitan archivos ground truth que representan una diferencia en cómo TensorFlow numera las categorías de salida.

Debido a la pequeña cantidad de imágenes en el set de ejemplo, el comando `dla_benchmark` establece `-niter=8` en esta sección para el set. Esta configuración limita el número de inferencias ejecutadas por el FPGA AI Suite IP.

Para mejorar el rendimiento de las pruebas, el demo `dla_benchmark` genera valores de imagen aleatorios si requiere

más imágenes de entrada que las que están disponibles en el set de ejemplo.

Copiando los gráficos compilados a la tarjeta SD

Para copiar los archivos de demostración requeridos a la carpeta `/home/root/resnet-50-tf` en la tarjeta SD debemos seguir los siguientes pasos.

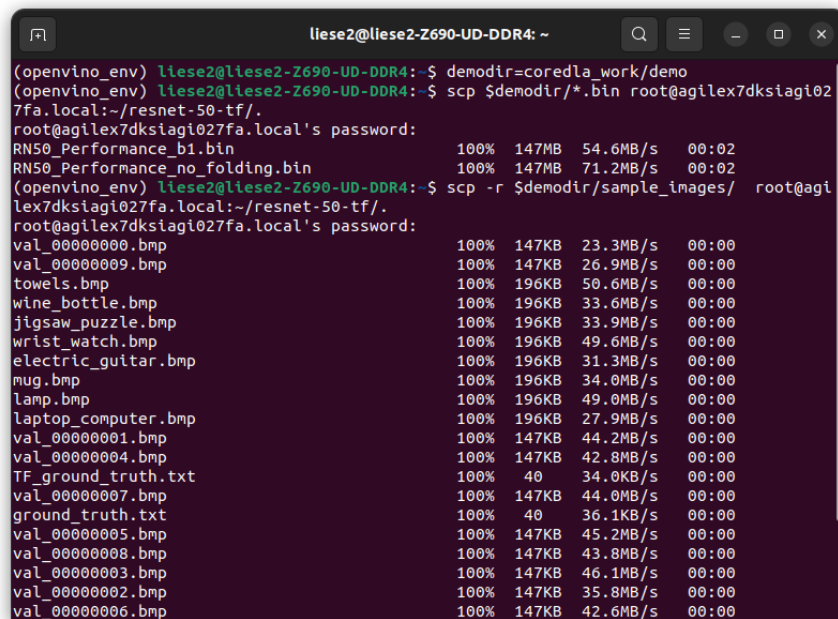
1. En la terminal serial (Minicom), se crea los directorios para recibir la información del modelo y las imágenes de muestra.

```
mkdir ~/resnet-50-tf
```

2. Dentro de la terminal de la estación de trabajo, se usa el comando de copia segura (`scp`) para transcribir la información a la terminal:

```
demodir=$COREDLA_WORK/demo
scp $demodir/*.bin root@agilex7dksiagi027fa.local:~/resnet-50-tf/.
scp -r $demodir/sample_images/ root@agilex7dksiagi027fa.local:~/resnet-50-tf/.
scp /opt/intel/fpga_ai_suite_2024.3/dla/example_architectures/AGX7_Performance.arch root@
agilex7dksiagi027fa.local:~/resnet-50-tf/.
scp /opt/intel/fpga_ai_suite_2024.3/dla/build_os.txt root@agilex7dksiagi027fa.local:~/
resnet-50-tf/../../app/
```

3. En la terminal ejecutar el comando `sync` para asegurar que los datos sean ingresados al disco.



```
liese2@liese2-Z690-UD-DDR4: ~
(openvino_env) liese2@liese2-Z690-UD-DDR4:~$ demodir=coredla_work/demo
(openvino_env) liese2@liese2-Z690-UD-DDR4:~$ scp $demodir/*.bin root@agilex7dksiagi027fa.local:~/resnet-50-tf/.
root@agilex7dksiagi027fa.local's password:
RN50_Performance_b1.bin          100% 147MB 54.6MB/s 00:02
RN50_Performance_no_folding.bin 100% 147MB 71.2MB/s 00:02
(openvino_env) liese2@liese2-Z690-UD-DDR4:~$ scp -r $demodir/sample_images/ root@agilex7dksiagi027fa.local:~/resnet-50-tf/.
root@agilex7dksiagi027fa.local's password:
val_00000000.bmp                100% 147KB 23.3MB/s 00:00
val_00000009.bmp                100% 147KB 26.9MB/s 00:00
towels.bmp                      100% 196KB 50.6MB/s 00:00
wine_bottle.bmp                 100% 196KB 33.6MB/s 00:00
jigsaw_puzzle.bmp               100% 196KB 33.9MB/s 00:00
wrist_watch.bmp                 100% 196KB 49.6MB/s 00:00
electric_guitar.bmp             100% 196KB 31.3MB/s 00:00
mug.bmp                         100% 196KB 34.0MB/s 00:00
lamp.bmp                       100% 196KB 49.0MB/s 00:00
laptop_computer.bmp             100% 196KB 27.9MB/s 00:00
val_00000001.bmp                100% 147KB 44.2MB/s 00:00
val_00000004.bmp                100% 147KB 42.8MB/s 00:00
TF_ground_truth.txt             100% 40    34.0KB/s 00:00
val_00000007.bmp                100% 147KB 44.0MB/s 00:00
ground_truth.txt                100% 40    36.1KB/s 00:00
val_00000005.bmp                100% 147KB 45.2MB/s 00:00
val_00000008.bmp                100% 147KB 43.8MB/s 00:00
val_00000003.bmp                100% 147KB 46.1MB/s 00:00
val_00000002.bmp                100% 147KB 35.8MB/s 00:00
val_00000006.bmp                100% 147KB 42.6MB/s 00:00
```

Figura 1.24: Salida al usar las primeras dos instrucciones del comando `scp`.

```

lieese2@lieese2-Z690-UD-DDR4: ~
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ ls /opt/intel/fpga_ai_suite_2024.3/dla/example_architectures/
A10_FP16_Generic.arch      AGX7_Generic.arch
A10_FP16_Performance.arch  AGX7_LayoutTransform_Resnet.arch
A10_Generic.arch          AGX7_Performance.arch
A10_Performance.arch      AGX7_Performance_Glant.arch
A10_Small_NoSoftmax.arch  AGX7_Performance_Multilane.arch
A10_Small_Softmax.arch    AGX7_Small_NoSoftmax.arch
AGX5_FP16_Generic.arch    AGX7_Small_Softmax.arch
AGX5_FP16_Performance.arch AGX7_Streaming_Ddrfree.arch
AGX5_Generic.arch         AGX7_Streaming_Ddrfree_Multilane.arch
AGX5_Performance.arch     AGX7_Streaming_Ddrfree_Resnet18.arch
AGX5_Small_NoSoftmax.arch build_bitstreams.sh
AGX5_Small_Softmax.arch   README.txt
AGX7_FP16_Generic.arch    S10_Generic.arch
AGX7_FP16_Performance.arch S10_Performance.arch
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ scp /opt/intel/fpga_ai_suite_2024.3/dla/example_architectures/AGX7_Performance.arch root@agilex7dksiagi027fa.local:~/resnet-50-tf/.
root@agilex7dksiagi027fa.local's password:
AGX7_Performance.arch                                100% 2712      1.0MB/s   00:00
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$ scp /opt/intel/fpga_ai_suite_2024.3/dla/build_os.txt root@agilex7dksiagi027fa.local:~/resnet-50-tf/./app/
root@agilex7dksiagi027fa.local's password:
build_os.txt                                          100% 9          5.5KB/s   00:00
(openvino_env) lieese2@lieese2-Z690-UD-DDR4:~$

```

Figura 1.25: Salida al ejecutar los últimos dos comandos `scp`.

Verificando los drivers de la FPGA

Los drivers de la tarjeta se deberían de cargar cuando el HPS arranca, pero podemos verificar que los drivers de la tarjeta se inicializaron revisando que los archivos `uio` están listados en el directorio `/sys/class/uio` al ejecutar el siguiente comando:

```
ls /sys/class/uio
```

La salida deberá ser similar al siguiente ejemplo:

```
uio0 uio1 uio2
```

Si los drivers no están listados, actualiza los módulos al ejecutar el siguiente comando y después revisa de nuevo si los drivers están cargados.

```
uio-devices restart
```

1.3.7 Ejecutando las aplicaciones de demostración

Ejecutando la aplicación de demostración del modo M2M

El modelo de flujo de datos M2M utiliza la aplicación de demostración `dla.benchmark`. El transmisor de datos S2M apoya ambos modelos de flujo de datos, el modelo M2M y el modelo S2M. Para ejecutar la inferencia en la tarjeta de desarrollo se deben seguir los siguientes pasos:

1. Abrir una conexión SSH a la tarjeta de desarrollo de la siguiente manera:

- Abrir una nueva terminal
- Ejecutar el siguiente comando (en caso de que pida contraseña ingresar [agilex7](#)):

```
ssh root@agilex7dksiagi027fa.local
```

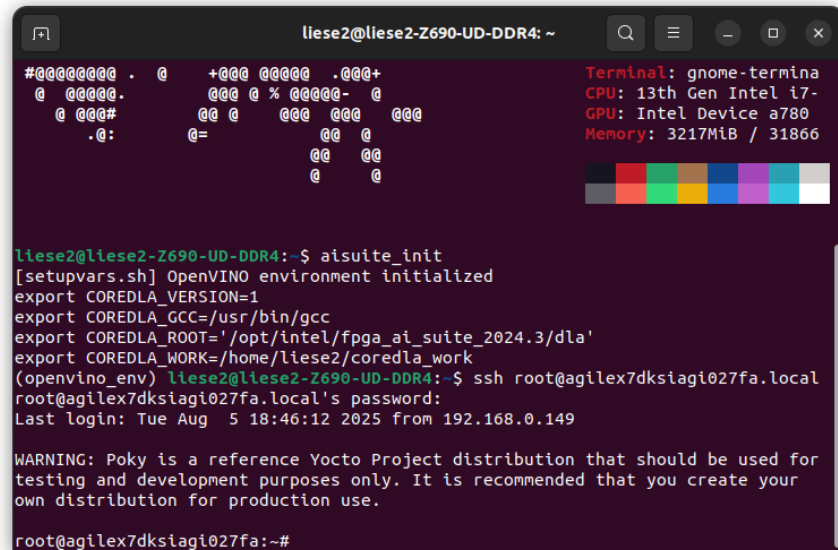
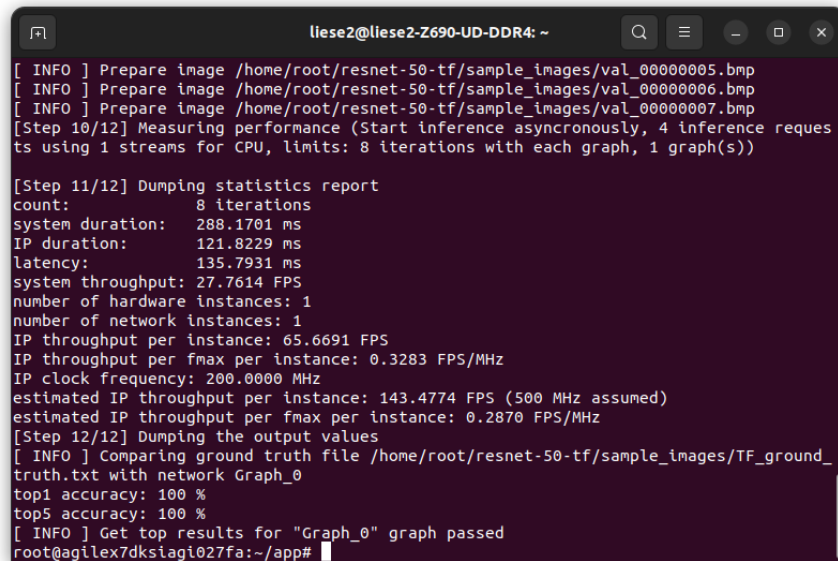


Figura 1.26: Conexión SSH con la tarjeta de desarrollo.

2. En la terminal con la conexión SSH, ejecutar los siguientes comandos:

```
export compiled_model=~/.resnet-50-tf/RN50_Performance_b1.bin
export imgdir=~/.resnet-50-tf/sample_images
export archfile=~/.resnet-50-tf/AGX7_Performance.arch
cd ~/app
export COREDLA_ROOT=/home/root/app
./dla_benchmark \
  -b=1 \
  -cm $compiled_model \
  -d=HETERO:FPGA,CPU \
  -i $imgdir \
  -niter=5 \
  -plugins_xml_file ./plugins.xml \
  -arch_file $archfile \
  -api=async \
  -groundtruth_loc $imgdir/TF_ground_truth.txt \
  -perf_est \
  -nreq=4 \
  -bgr
```

El comando `dla_benchmark` genera la siguiente salida:



```
liese2@liese2-Z690-UD-DDR4: ~
[ INFO ] Prepare image /home/root/resnet-50-tf/sample_images/val_00000005.bmp
[ INFO ] Prepare image /home/root/resnet-50-tf/sample_images/val_00000006.bmp
[ INFO ] Prepare image /home/root/resnet-50-tf/sample_images/val_00000007.bmp
[Step 10/12] Measuring performance (Start inference asynchronously, 4 inference requests using 1 streams for CPU, limits: 8 iterations with each graph, 1 graph(s))

[Step 11/12] Dumping statistics report
count: 8 iterations
system duration: 288.1701 ms
IP duration: 121.8229 ms
latency: 135.7931 ms
system throughput: 27.7614 FPS
number of hardware instances: 1
number of network instances: 1
IP throughput per instance: 65.6691 FPS
IP throughput per fmax per instance: 0.3283 FPS/MHz
IP clock frequency: 200.0000 MHz
estimated IP throughput per instance: 143.4774 FPS (500 MHz assumed)
estimated IP throughput per fmax per instance: 0.2870 FPS/MHz
[Step 12/12] Dumping the output values
[ INFO ] Comparing ground truth file /home/root/resnet-50-tf/sample_images/TF_ground_truth.txt with network Graph_0
top1 accuracy: 100 %
top5 accuracy: 100 %
[ INFO ] Get top results for "Graph_0" graph passed
root@agilex7dksiagi027fa:~/app#
```

Figura 1.27: Salida generada al ejecutar la aplicación de demostración del modo M2M.

Ejecutando la aplicación de demostración del modo S2M

Para ejecutar la aplicación de modo de demostración S2M se necesitan dos conexiones en la terminal de la tarjeta de desarrollo.

Para ejecutarlo, se siguen los siguientes pasos

1. Abrir dos conexiones SSH con la tarjeta de desarrollo (dos terminales)
2. Ejecutar el siguiente comando en ambas terminales:

```
ssh root@agilex7dksiagi027fa.local
```

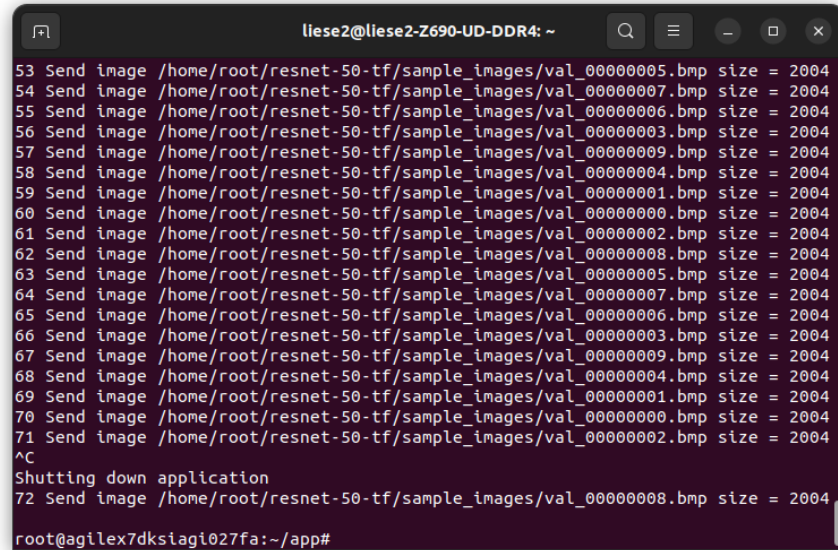
3. En una de las terminales ejecutar los siguientes comandos:

```
cd /home/root/app
export COREDLA_ROOT=~/.app
export LD_LIBRARY_PATH=.
./run_inference_stream.sh
```

4. En la otra terminal, ejecutar los siguientes comandos:

```
cd /home/root/app
./run_image_stream.sh
```

En la segunda terminal (donde se ejecutó el script `run_image_stream.sh`) se mostrará una salida similar a la siguiente:



```

lieese2@lieese2-Z690-UD-DDR4: ~
53 Send image /home/root/resnet-50-tf/sample_images/val_00000005.bmp size = 2004
54 Send image /home/root/resnet-50-tf/sample_images/val_00000007.bmp size = 2004
55 Send image /home/root/resnet-50-tf/sample_images/val_00000006.bmp size = 2004
56 Send image /home/root/resnet-50-tf/sample_images/val_00000003.bmp size = 2004
57 Send image /home/root/resnet-50-tf/sample_images/val_00000009.bmp size = 2004
58 Send image /home/root/resnet-50-tf/sample_images/val_00000004.bmp size = 2004
59 Send image /home/root/resnet-50-tf/sample_images/val_00000001.bmp size = 2004
60 Send image /home/root/resnet-50-tf/sample_images/val_00000000.bmp size = 2004
61 Send image /home/root/resnet-50-tf/sample_images/val_00000002.bmp size = 2004
62 Send image /home/root/resnet-50-tf/sample_images/val_00000008.bmp size = 2004
63 Send image /home/root/resnet-50-tf/sample_images/val_00000005.bmp size = 2004
64 Send image /home/root/resnet-50-tf/sample_images/val_00000007.bmp size = 2004
65 Send image /home/root/resnet-50-tf/sample_images/val_00000006.bmp size = 2004
66 Send image /home/root/resnet-50-tf/sample_images/val_00000003.bmp size = 2004
67 Send image /home/root/resnet-50-tf/sample_images/val_00000009.bmp size = 2004
68 Send image /home/root/resnet-50-tf/sample_images/val_00000004.bmp size = 2004
69 Send image /home/root/resnet-50-tf/sample_images/val_00000001.bmp size = 2004
70 Send image /home/root/resnet-50-tf/sample_images/val_00000000.bmp size = 2004
71 Send image /home/root/resnet-50-tf/sample_images/val_00000002.bmp size = 2004
^C
Shutting down application
72 Send image /home/root/resnet-50-tf/sample_images/val_00000008.bmp size = 2004
root@agilex7dkslagi027fa:~/app#

```

Figura 1.28: Flujo de datos que se da al ejecutar la aplicación de demostración.

Los datos resultantes se guardarán en el archivo `results.txt`, se puede ver el contenido de dicho archivo ejecutando el siguiente comando en terminal:

```
cat results.txt
```

Resolviendo errores de las aplicaciones de demostración

Si se produce algún error parecido al siguiente mientras se está ejecutando la aplicación M2M o S2M, se debe revisar que todas las DIMMs de la tarjeta están instaladas de forma segura:

```
altera-msgdma ff200000.msgdma: dma_sync_wait: timeout! [ 251.812846] DMA Failed
```

1.3.8 Realizando inferencias sin una FPGA

El modelo de emulación de software es un modelo de software en C++, representa en software al FPGA AI Suite IP que se usa para emular de manera precisa ² el comportamiento del hardware.

Para usar el software de modelo de emulación:

1. Construye el entorno de ejecución con los siguientes comandos:

²diferencias mínimas entre el software de emulación y el hardware que típicamente resultarán en una diferencia de menos de dos unidades de menor precisión

```
cd $COREDLA_WORK/runtime
rm -rf build_Release
./build_runtime.sh -target_emulation
```

2. Ejecuta la inferencia con las opciones `-niter=1` y `-nireq=1` (debido a que el modelo de software es lento) con los siguientes comandos:

```
imagedir=$COREDLA_WORK/demo/sample_images
curarch=$COREDLA_ROOT/example_architectures/AGX7_Performance.arch
gnd=$imagedir/TF_ground_truth.txt
cd $COREDLA_WORK/runtime/build_Release/dla_benchmark
./dla_benchmark \
  -b 1 `# Run only a single image` \
  -niter 1 `# Run only a single image` \
  -nireq 1 `# Running emulator: so -nireq=1` \
  -m $modeldir/resnet-50-tf/FP32/resnet-50-tf.xml \
  `# Same as when running on hardware - specify the graph` \
  -d HETERO:FPGA,CPU \
  `# Same as when running on hardware - \
  use FPGA if possible, fallback to CPU` \
  -i $imagedir \
  `# Same as when running on hardware - specify image directory` \
  -arch_file $curarch \
  `# Same as when running on hardware - specify .arch file` \
  -dump_output \
  `# Dump output result.txt file` \
  -plugins emulation \
  `# Use the software emulator` \
  -groundtruth_loc $gnd
  `# Location of the ground truth file for $imagedir`
```

1.4 Demostración de transmisión Streaming-to-Memory (S2M)

Un caso típico de uso del FPGA AI Suite IP³ es la ejecución de inferencias usando datos de entrada en tiempo real. Por ejemplo, los datos en tiempo real pueden venir de una fuente de video como HDMI IP y transmitirla al FPGA AI Suite IP para realizar clasificación de imágenes para cada frame.

Por simplicidad, la demostración S2M solamente simula una fuente de video en tiempo real. La demostración de transmisión consta de las siguientes aplicaciones que se ejecutan en el dispositivo SoC objetivo:

- **streaming_inference_app**: Esta aplicación carga y ejecuta una red y captura los resultados.
- **image_streaming_app**: Esta aplicación carga los archivos bitmap desde un directorio de la tarjeta SD y los manda de forma continua al EMIF (Intefaz de Memoria Externa), simulando una fuente de video continua.

³El FPGA AI Suite IP es un bloque de hardware (IP Core) que Intel provee para implementar aceleración de inferencia de redes neuronales directamente dentro de FPGAs. Este bloque recibe las imágenes o datos en el formato requerido, los procesa según el modelo cargado y entrega la salida al sistema.

Las imágenes se mandan a través de un adaptador que reordena y convierte el formato (layout transform IP), que mapea las imágenes entrantes y las almacena en un formato específico (frame buffer encoding) que es requerido por el FPGA AI Suite IP.

Hay un módulo llamado controlador de transmisión, que se ejecuta en un microcontrolador Nios V, que controla la organización de las imágenes de entrada al FPGA AI Suite IP.

La aplicación `streaming_inference_app` crea solicitudes de inferencia de OpenVINO. Cada solicitud de inferencia es memoria localizada en el EMIF para los buffers de entrada y salida. Esta información se manda al controlador de transmisión cuando las solicitudes de inferencia son enviadas para ejecución asíncrona.

En el modo de ejecución, el controlador de transmisión espera por los buffers de entrada que llegan desde la aplicación `image_streaming_app`. Cuando el buffer llega, el controlador de transmisión programa el FPGA AI Suite IP con los detalles del buffer de entrada recibido, el cual dispara el FPGA AI Suite IP para que ejecute una inferencia.

Cuando una inferencia se completa, un registro contador se incrementa dentro del FPGA AI Suite IP CSRs (Registros de Control y Estado). Este contador es monitoreado por la solicitud de inferencia que se está ejecutando en la aplicación `streaming_inference_app`, y se marca como completada cuando el incremento se detecta. Entonces es cuando se llama al buffer de salida desde el EMIF y la porción de la inferencia del FPGA AI Suite IP se completa.

Dependiendo del modelo usado puede que haya procesamiento de la salida extra por parte del plugin OpenVINO HETERO y el plugin OpenVINO Arm CPU. Después de que se ha procesado la red completa, se hace un llamado a la aplicación para indicar que la inferencia se ha completado.

La aplicación realiza algunos procesamientos posteriores en el buffer para generar los resultados y reenviar la misma solicitud de inferencia de vuelta a OpenVINO, el cual deja que el controlador de transmisión use el mismo bloque de memoria entrada/salida de nuevo.

1.4.1 Configuración de las aplicaciones de demostración de transmisión

Las dos aplicaciones de demostración de transmisión, `streaming_inference_app` y `image_streaming_app`, están construidas como parte del entorno de ejecución y se encuentran en la imagen de la tarjeta SD para los dispositivos objetivos en el directorio `/home/root/app/`.

1.4.2 Ejecución de la demostración de transmisión

Para realizar la ejecución, se necesitan dos terminales conectadas al dispositivo objetivo, una por cada aplicación de transmisión. Una puede ser una terminal serial y la otra puede ser una conexión SSH desde la estación de trabajo, o bien, ambas pueden ser conexiones SSH.

La aplicación `streaming_inference_app`

La aplicación `streaming_inference_app` es una aplicación basada en OpenVINO. Carga una red ResNet50 precompilada, luego crea solicitudes de inferencia que son ejecutadas de forma asíncrona por el FPGA AI Suite IP.

Los tensores resultantes son capturados desde el EMIF usando el controlador mSGDMA. El procesamiento posterior requerido en el software involucra convertir los tensores de salida a punto flotante, asignar el valor de la

clasificación de imagen apropiada, ordenar los resultados y seleccionar los 5 mejores resultados de clasificación.

Por cada inferencia, los resultados son mostrados en terminal, y los resultados de cada inferencia hasta la número 1000 son capturados en un archivo `results.txt` en el directorio de la aplicación.

La aplicación depende de las siguientes librerías compartidas. El sistema cargado agrega estas librerías al directorio `/home/root/app` en la imagen de la tarjeta SD, junto con el binario de la aplicación y el archivo plugin `.xml` que define los plugins disponibles para OpenVINO.

- `libhps_platform_mmd.so`
- `libngraph.so`
- `libinference_engine.so`
- `libinference_engine_transformations.so`
- `libcoreDLAHeteroPlugin.so`
- `libcoreDlaRuntimePlugin.so`

Además, es necesario un archivo binario compilado de la red (el cual describe la parametrización del FPGA AI Suite IP) para ejecutar las inferencias. Estos archivos fueron copiados al directorio `/home/root/resnet-50-tf`.

Antes de ejecutar la aplicación se debe establecer la variable de entorno `LD_LIBRARY_PATH` para definir la localización de las librerías compartidas.

```
cd /home/root/app
export COREDLA_ROOT=/home/root/app
export LD_LIBRARY_PATH=.
```

Se puede usar el argumento `--help` de `streaming_inference_app` para mostrar el uso del comando:

```
./streaming_inference_app --help
```

Lo cual nos muestra las opciones que se pueden agregar a la línea de comando:

```
Usage:
    streaming_inference_app -model=<model> -arch=<arch> -device=<device>
Where:
    <model>      is the compiled model binary file, eg /home/root/resnet-50-tf/
RN50_Performance_no_folding.bin
    <arch>       is the architecture file, eg /home/root/resnet-50-tf/
A10_Performance.arch
    <device>     is the OpenVINO device ID, eg HETERO:FPGA or HETERO:FPGA,CPU
```

Después se puede iniciar la aplicación de la siguiente manera:

```
./streaming_inference_app \
```

```
-model=/home/root/resnet-50-tf/RN50_Performance_no_folding.bin \
-arch=/home/root/resnet-50-tf/AGX7_Performance.arch \
-device=HETERO:FPGA
```

Notese que dentro del mismo directorio se incluye un script llamado `run_inference_stream.sh`, el cual ejecuta el comando anterior. Además, el adaptador de formato no soporta folding en el buffer de entrada. Para la transmisión, se deben usar modelos que han sido compilados con el comando `dla_compiler` con la opción `--folding_option=0` especificada en la línea de comando.

La aplicación `image_streaming_app`

La aplicación `image_streaming_app` carga imágenes desde la tarjeta SD, programa el adaptador de formato y luego transfiere los buffers mediante la interfaz de transmisión mSGDMA al dispositivo. Los buffers son enviados a intervalos regulares a una frecuencia establecida por una de las opciones de la línea de comandos. Los buffers son enviados de manera continua hasta que el programa se detenga con `Ctl+C`.

La aplicación `image_streaming_app` solo trabaja con archivos `.bmp` de 224x224 pixeles. Los archivos `.bmp` pueden tener el formato de 24 o de 32 bits por pixel. Si son de 24 bits los buffers son modificados para hacerlos de 32 bits por pixel. Este formato es el esperado por la entrada del adaptador de formato.

Se puede usar el argumento `--help` de `image_streaming_app` para mostrar el uso del comando:

```
./image_streaming_app --help
```

Lo cual nos muestra las opciones que se pueden agregar a la línea de comando:

```
Usage:
  image_streaming_app [Options]

Options:
  -images_folder=folder      Location of bitmap files. Defaults to working folder.
  -image=path                Location of a single bitmap file for single inference.
  -send=n                    Number of images to stream. Default is 1 if -image is
  set, otherwise infinite.
  -rate=n                    Rate to stream images, in Hz. n is an integer. Default
  is 30.
  -width=n                   Image width in pixels, default = 224
  -height=n                  Image height in pixels, default = 224
  -c_vector=n                C vector size, default = 32
  -blue_variance=n           Blue variance, default = 1.0
  -green_variance=n          Green variance, default = 1.0
  -red_variance=n            Red variance, default = 1.0
  -blue_shift=n              Blue shift, default = -103.94
  -green_shift=n             Green shift, default -116.78
  -red_shift=n               Red shift, default = -123.68
```

De igual manera que la aplicación de transmisión de inferencia, en el mismo directorio se incluye un script llamado

`run_image_stream.sh` que llama al comando `image_streaming_app` con una frecuencia de 50 *Hz* y opciones por defecto del adaptador de formato.

Una vez que ambas aplicaciones se están ejecutando, la aplicación `streaming_inference_app` muestra los resultados de salida en la terminal y en el archivo `results.text`.

Un ejemplo de la salida de la aplicación `streaming_inference_app` sería parecido al siguiente:

El archivo `results.txt` contendría resultados como los siguientes: